Software Development 1.1

- Explore fundamental software development steps used by programmers when designing software Including:
  - Requirements definition
  - Determining specifications
  - Design
  - Development
  - Integration
  - Testing and debugging
  - Installation
  - Maintenance
- Research and evaluate the prevalence and use of online code collaboration tools

In the past software development has been accomplished in distinct stages, where each stage must be completed before the next is commenced. Altering requirements during development was difficult and costly. Unfortunately, due to the long development times, many requirements had substantially changed before the product had been released for use. Software developers had to acknowledge this problem and respond by altering their approach to software development. Faster, more reactive approaches were required. The creation of software tools to assist in speeding up and reducing the formality of the task began to emerge with the introduction of the Rapid Application Development and Prototyping and more recently the Agile Approach to software development.

In this chapter we will be looking at two Software Development Approaches - The Structured Development Approach and the Agile Development Approach.

Structured or Waterfall Development Approach

The structured approach to software development consists of distinct stages. Each stage needs to be completed before the next stage can commence. This is necessary as teams of developers with varying skills and responsibilities are involved in the development process. For example, the coding of the solution cannot commence until the solution has been thoroughly planned in its entirety. The personnel coding the solution are often different to those who plan the solution.

The traditional stages of the Structured Approach to software development include:

- Requirements definition
- Determining specifications
- Design
- Development
- Integration
- Testing and debugging
- Installation
- Maintenance

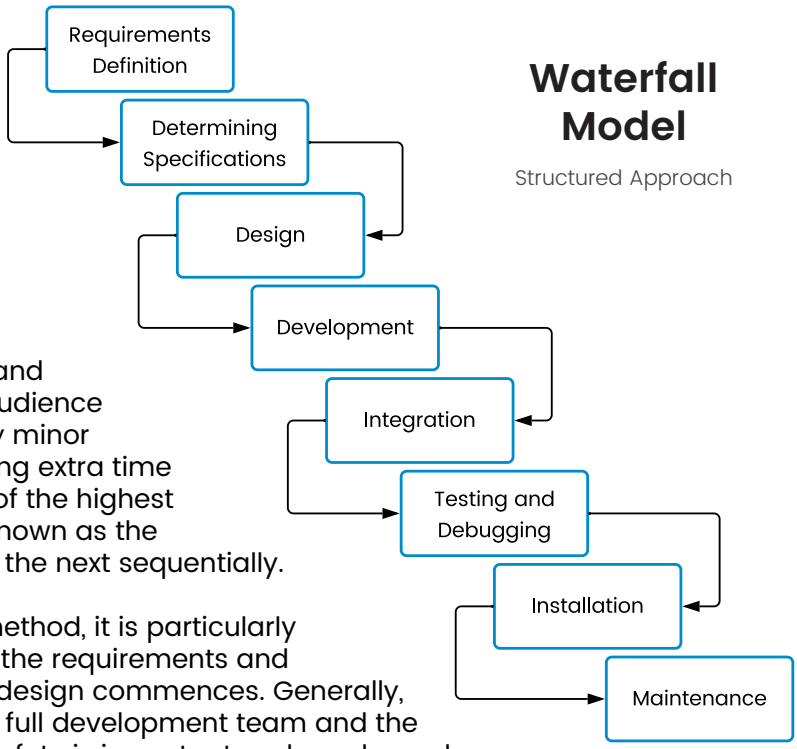


Fig 1.1.1

This approach is generally used for large-scale projects where performance and reliability are vital requirements. A large audience will use the final product so even relatively minor errors may prove costly. It is worth spending extra time and money to ensure the final product is of the highest quality. This structured approach is also known as the Waterfall approach as each step leads to the next sequentially.

Because of the structured nature of this method, it is particularly suited to software development where all the requirements and specifications can be defined before any design commences. Generally, these are large-scale projects requiring a full development team and the timeframe is long with a large budget. If safety is important and or a bespoke solution is required, then the Structured Approach is likely the most suitable.

SCENARIO & DISCUSSION

Scenario: Mining Company Software
A large mining company wants to develop software to manage the work, health and safety of its employees. There is much government legislations in place to protect worker safety. This software will help to protect the workers by logging the hours worked and breaks taken to ensure employees are able to work at their best. There is a large budget, and a software development company has been employed to manage the production of the software from beginning to end.

Discussion: Discuss the suitability of the Structured Approach for this project? Consider both advantages and disadvantages.

CLASS DISCUSSION

Discuss: Defining the problem
Defining the problem precisely is particularly vital when using the structured approach. Why do you think this is the case? Why might it be difficult to accomplish this task? Discuss.

Agile Development Approach

Agile development has emerged in response to the 'ad-hoc' reality of many software development projects. This approach places emphasis on the team developing the system rather than following the pre-defined structured development stages. Agile methods remove the need for detailed requirements and complex design documentation. Rather, they encourage cooperation and teamwork.

Agile methods are particularly well suited to user-centred software development and software applications that are modified regularly such that they evolve and are updated over time. Many mobile apps are developed using this approach. This can be seen in the frequency of updates and the nature of app development itself.

ad-hoc: Creating software without any formal guidelines or process.

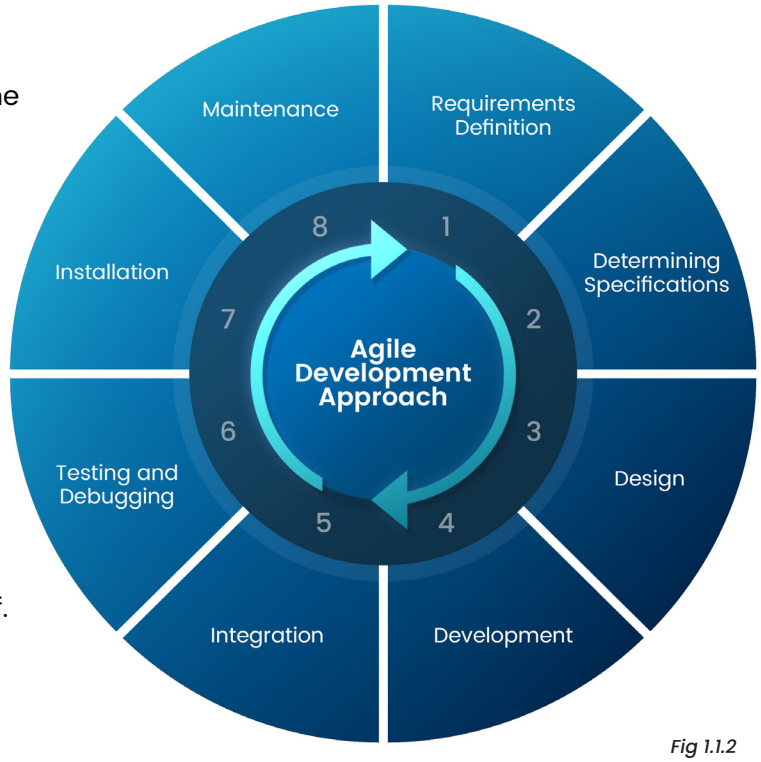


Fig 1.1.2

Agile development is characterised by small teams of developer. It is preferable for one team member to be a knowledgeable and experienced user. Small teams are better able to share ideas and work on solutions together. Larger teams tend to break into smaller groups – for agile methods to be a success everyone must be an equal member with a clear shared purpose. Often the team members are multi-skilled so all are actively contributing.

Typical characteristics of an agile software development approach include:

- **Agility:** Speed of getting a working solution to market or to users. Basic functionality is included initially so operational software can be released as soon as possible.
- **Adaptive:** Interaction within the team and users which allows the solution to be selectively refined throughout the development process.
- **Progressive:** Working versions of the software are regularly delivered. Each version adds the next most important functions.
- **Collaborative:** The development team and clients collaborate closely throughout the development. Often a client representative or user is part of the team. The needs and ongoing feedback of the client and users drives the direction of the development.

Let us work through the typical sequence of activities occurring during agile development. Initially the general nature of the problem is determined and the development team is formed. The team first meet to create a basic plan and a general design for the software – only minimal detail at this stage, just enough to get started. The basic idea is to only plan, design and document details as they're actually needed. Often a simple whiteboard is used to sketch out the general design. The team then gets straight to the task of creating an initial solution. As this occurs they informally consult and negotiate with each other. The user team member is always present to answer questions, make suggestions and generally ensure the solution will be workable in practice.

Once an initial, yet simplified, solution is implemented it is immediately tested, evaluated and then released for use. This means a working version of the software is actually being used by real users – usually the client but it could be a sample of users or even globally to all users via the web. The users see exactly what has been achieved, can provide feedback and make suggestions about further additions. In effect we have entered a new mini "requirements definition" phase.

The team again meets informally to discuss the next iteration to be developed. The design incorporates feedback from users together with their own ideas. They then go straight to work coding this next version of the solution. The solution is again thoroughly tested and evaluated before being released. This process repeats many times with each iteration implementing further functionality and detail. Typically, a single iteration takes just weeks or even days. Each team meeting is short, maybe just an hour or so, whilst coding and testing consumes the majority of the development time.

In many Agile approaches the focus is on the user and what they want the software to actually do. Often "user stories" are created by each different type of user. These user stories are prioritised and each becomes the focus of a development iteration. Each user story involves one or more actors (often users, such as customers or employees) and is refined using suitable system models.

Currently UML (Unified Modelling Language) is the industry standard for modelling. The UML includes a variety of diagrams and notations for modelling different aspects of software systems. For many teams, user stories are simply written on slips of paper or on a whiteboard, whilst other teams may utilise software tools to produce them. During design and development new user stories will emerge, some will be removed, others will be reprioritised and others may be split into multiple user stories.

RESEARCH TASK

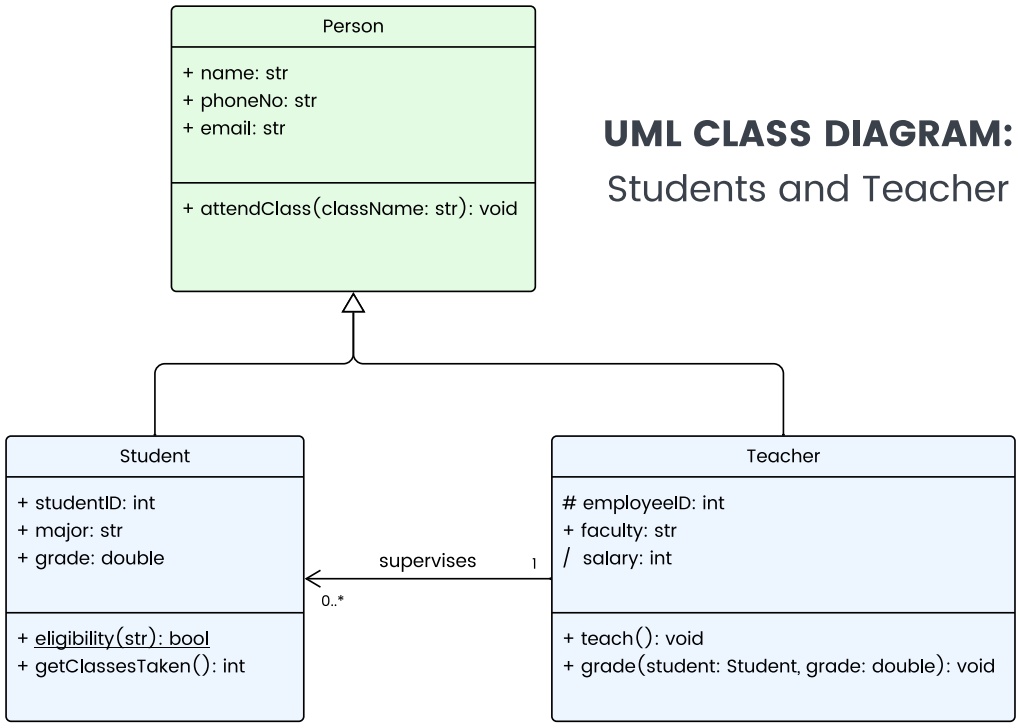


Fig 1.1.3

Research: UML Diagrams
Different diagrams, models and notations within the UML (Unified Modelling Language) are integral to many Agile approaches. Research the different diagrams, models and notations that form part of the UML.

When developing software it's all the small details that combine to form the total solution. Agile methods are a response to the reality that intricate details are difficult to specify accurately in advance. Each part of a software solution relies heavily on many other related parts.

Until the related parts exist, it is wasteful to continue designing. Much of the design will prove unworkable and will need to be redesigned or significantly altered. Compare this to the traditional structured approach where specific and intricate detail is created well in advance.

One significant issue with agile methods is how to construct agreements when outsourcing the development. Traditionally a strict set of detailed requirements, together with the total cost and time for completion is negotiated. When using agile methods no detailed specifications exist – they emerge and change during development. A common solution to this dilemma is to fix the budget and time and allow the specifications to change. Once the budget and time is exhausted then the current solution becomes the final solution. To enter into such agreements requires significant trust to be established between the client and developer. The client stands to gain, as they are heavily involved throughout the development process and hence are more likely to receive a final product that better meets their actual and current requirements.

RESEARCH TASK

The following manifesto was written in 2001 by 17 diverse software development experts during the formation of the Software Development Alliance, better known as the Agile Alliance (agilealliance.org). The manifesto defines four core values to encourage better ways of developing software.

The Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Research 1: The Agile Alliance
The Agile Alliance (agilealliance.org) provides a variety of resources for those using or intending to use an Agile approach to software development. Research the initiatives and resources provided by the Agile Alliance. In particular, ensure you read their "Twelve Principles of Agile Software" which expand upon the above manifesto.

Research 2: Agile Methods
There are a variety of different versions of the agile approach. For example, Scrum, Extreme Programming, Crystal, and Lean software development. Research a variety of agile methods and summarise the major features.

CLASS DISCUSSION

Research & Discuss: Test Driven Development
Many agile methods include a test-driven development (TDD) methodology. Essentially tests are always coded before writing any actual code. The tests will of course fail before the actual code is written and will pass once the code is correct. Numerous small tests are added to the test suite as development progresses and all these tests are rerun each time new code is added to the project. Research and discuss advantages of TDD.

Requirements Definition

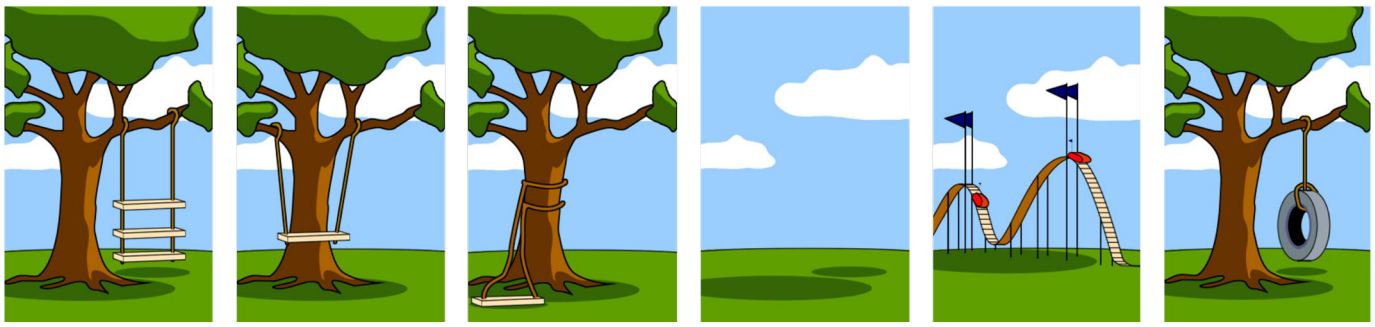
To further understand the structured approach, we will begin by investigating the requirements definition process in greater detail. In many ways this initial stage is the most important and is sometimes overlooked by large software development companies as trivial. Studies have shown that some 30% of software projects are abandoned well into the development cycle. Many of these cancelled projects are well over budget and were not feasible from the outset. Software developers must take steps to carefully define and understand the requirements before moving onto the design and development phases.

The requirements of the problem must be understood precisely. Omissions at this stage will prove the most expensive to correct during later stages. System analysts are experts in determining and defining requirements. The set of requirements should clearly and precisely define each aspect of the problem. These requirements will later be used to test the success of the solution.

Key Term: Requirements
Features, properties, or behaviours a system must have to achieve its purpose. Each requirement must be verifiable.

During this first stage of the software development process the requirements of the system are determined and agreed upon with the client. It is vital that the users' needs, requirements and expectations are fully understood before any software product is designed.

Requirements need to be correctly and accurately defined so that a project can be successful. Requirements are features, properties or behaviors a system must have in order to achieve its purpose. Each requirement must be verifiable.



How the customer explained it | How the project leader understood it | How the programmer coded it | How the project was documented | How the customer was billed | What the customer really needed

CLASS DISCUSSION

Discuss: Structured Approach Requirements
When the structured approach is being used design does not commence until all requirements and specifications have been determined. Can all the requirements and specifications be known at this stage? Discuss.

Business Systems Analysts

The requirements of the problem must be understood precisely. This is the job of a Business System Analyst. Business System analysts are experts in determining and defining requirements. The set of requirements should clearly and precisely define each aspect of the problem. These requirements will later be used to test the success of the solution.

Key Term: Business Systems Analyst
A person who analyses systems, determines requirements and designs new information systems.

Business System analysts often commence by compiling a list of required outputs. These outputs are then examined to determine the required inputs. The system analyst is then able to create models showing the flow of data through the system from inputs through the various processes to the final outputs.

Determining requirements involves consideration of all aspects that can affect the operation of the final system. Existing systems should be studied to ensure the new system will accomplish all existing requirements. New requirements will then be added. All elements of the system should be considered. There is no point designing a product that cannot operate on the available hardware. The potential users of the system should be consulted to determine their needs and requirements.

RESEARCH TASK

Research 1: Role of Business Systems Analysts
Research the role of the Business Systems Analyst in the development of Software.

Research 2: Roles In Software Development
Personnel with specific skills are used when large products are developed using the structured approach. Describe the tasks performed by system analysts, programmers, software testers.

Meeting the needs of the Client

One of the most important considerations when determining requirements are the needs of the client. Clients often come up with very general statements such as 'we need to monitor sales across each of our stores' or are expressed in non-specific, even emotive terms. For example, 'we need to improve customer relations', or 'we want to improve our turnaround times'. The requirements need to refine such requests into achievable and verifiable statements.

A few tools and techniques are available for accomplishing the analysis.

- **Surveys** - Completed by all key personnel. For example, managers, IT personnel, users and clients. Surveys are an excellent tool for gathering information from large numbers of people, however they often limit the detail and explanations participants are able to provide.
- **Interviews** - Personal interviews will allow participants to more freely express their needs. Detailed responses and explanations are possible, however performing interviews is labour intensive and therefore expensive.
- **Time management studies** - considers how much time is really spent on each specific function. Actually watching and recording the time, nature and order of tasks as they are performed will often highlight priority areas.
- **Business analysis** - Examining different aspects of a business' activities in search of areas where improvement can be made. Often such analysis will include a cost-benefit analysis to ensure a profitable return on the software investment is likely.

User Stories

In many agile software development methodologies, projects are organised using a hierarchical structure that includes initiatives, epics, and user stories. Initiatives represent strategic goals, guiding the overall direction of development, while epics serve as high-level features encompassing multiple user stories. A user story is a concise, user-focused description of features or functionality from the user's perspective.

Typically, user stories are created using a template:

"**As a** [role],
I want [goal],
so that [benefit/value]".

For example, "As an order clerk, I want to see all previous orders for a customer using their customer ID, so I can suggest products that they may like to add to their order.

User stories can be used as feature requests, a prioritised list maintained by the developers for possible implementation when time or budget allows.

When using agile development, each development iteration is commonly known as a sprint. Before each sprint, the team selects user stories from the prioritized list to work on. At the end of each sprint, the team showcases completed user stories to stakeholders for feedback and review. Once stakeholders are satisfied, the product increment is released, marking the end of a sprint cycle.

By using user stories, agile teams maintain a focus on delivering directly to users, adapt to changing requirements, and encourage collaboration among team members, users and all stakeholders throughout the development process.

USER STORIES:
e-commerce company

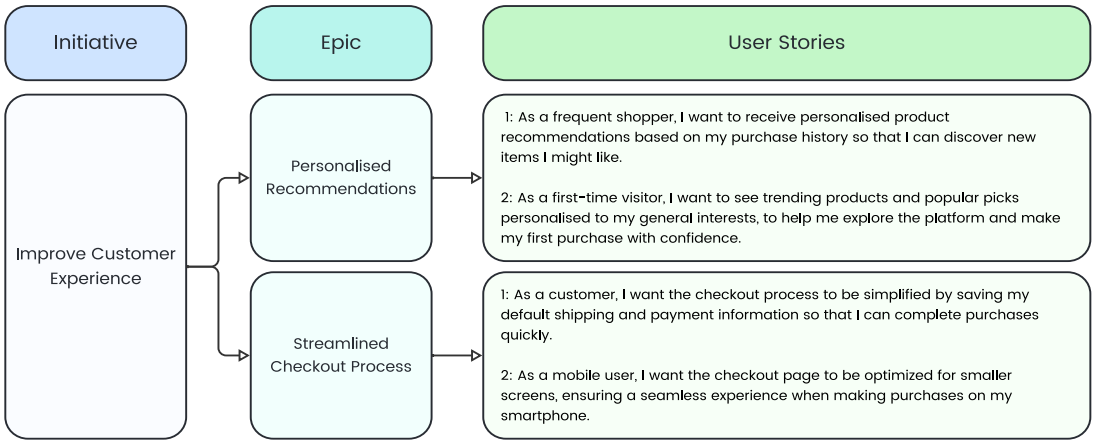


Fig 1.1.4

CLASS DISCUSSION

Research: Market Demand

Many people have great ideas for new products. Developing a new product of any type without establishing a need for the product is like putting the proverbial cart before the horse. Many great inventions have remained as ideas because no market or need exists for the product. Conversely, many fine products appear which fail economically because they don't meet the needs of the marketplace.

1. Brainstorm products and inventions that have not been a success.
2. How could the potential success or failure of new product ideas be predicted?
3. Consider 'Go Fund Me" style projects and how they assist in determining the likelihood of success of a product.

CLASS DISCUSSION

Context: Mail Order Business Requirements

A small mail order business has grown to a stage where it is no longer viable to use a manual ordering system.

The business has established a list of needs as follows:

1. Orders must be processed more efficiently.
2. Overdue accounts need to be dealt with more effectively.
3. Items on backorder should always be filled in before new orders.
4. Customers need to feel confident their personal details will remain confidential.
5. Personal attention to customers must be maintained.

Discuss: Mail Order Business Requirements

After further discussion and analysis, a set of requirements is defined as follows:

1: 90% of orders will leave the premises within 48 hours. Customers whose orders are not filled in within this time are contacted by phone.

2: Overdue accounts are generated automatically after 30 days.

3: Accounts over 60 days are contacted personally. If a resolution is not found, they are referred to a collection agency.

4: Stock to be entered into the computer system before it is placed in the warehouse.

5: Backorders are always checked before items are allocated to orders.

6: Password protection on the customer details file.

7: An email or SMS is sent to each customer after any progress on their order.

Discuss the relationship between each requirement and the needs. Is each of the requirements verifiable?

Environment and Boundary of the System

Boundaries define the limits of the problem or system to be developed. Anything outside the system is said to be a part of the environment. The system interacts with its environment via an interface. Input and output to and from the system takes place via an interface. For example, the keyboard provides an interface that allows humans to input data into a computer system. An internet service provider (ISP) provides an interface between computers and the internet.

Key Term: Environment
The circumstances and conditions that surround an information system. Everything that influences or is influenced by the system.

It is vital to determine the boundaries of a problem to be solved. Determining the boundaries is, in effect, determining what is and what is not a part of the system.

Key Term: Boundary
The delineation between a system and its environment. The boundary defines what is and isn't part of the environment.

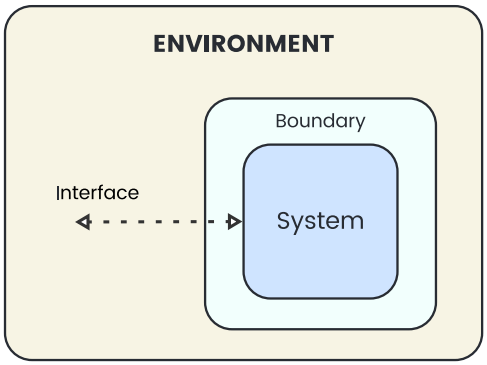


Fig 1.1.5

Items that are not part of the system, that is items that are in the environment, need to be considered if they have an influence on the system. For items in the environment to influence the system there must be an interface between the system and the environment. Items in the environment can affect the system, however the system cannot alter its environment.

When defining a requirement, it is important to define the boundaries of the system so that the customer has realistic expectations of the limits or scope of the new system.

Determining Specifications

The terms Requirements and Specifications are used in different contexts and are routinely combined or even interchanged. In this course defining requirements comes before specifications are determined. Each is a separate process. In general terms, think of requirements as business requirements; they are expressed from the perspective of the business management and staff. Specifications are written for the developers; think technical specifications. Further, often specifications are split into groups comprised of functional and non-functional specifications.

CLASS DISCUSSION

Research & Discussion: Software Requirements Specification
The term Software Requirements Specification (SRS) is used in many references. Research and discuss examples of SRS documents and compare with the distinct "requirements definition" and "determining specifications" stages specified in the software engineering course syllabus.

Functional Specifications

As the term function implies, these have an input (data if you like) that is transformed into an output (information). For example, a specification "The system shall calculate and store the average of all raw assessment marks for every student studying each course". The raw assessment marks, tasks and the associated courses are the input data. This data is transformed (processed) and output as the set of averages for each course. A function has performed this process, hence this is a functional specification and it will later be coded to form a function within the software product. Functional specifications, like a maths function, take a set of one or more inputs and process them into a set of outputs, and just like a maths function, the inputs always generate the same outputs.

The set of functional specifications will be a refinement of the requirements, but expressed precisely, such that they are suitable for turning into a model, then an algorithm during the design phase and then coded as part of the development stage.

Non-functional Specifications.

As the name suggests, non-functional specifications do not transform inputs into outputs, however they are nevertheless critical to the success of the software project. Non-functional specifications are often referred to as design specifications. They determine the framework under which the developers will work and areas that influence the experience of the end users.

NON-FUNCTIONAL SPECIFICATIONS	
Developer's Perspective	**User's Perspective**
Data types	Interface design
Data structures	Appropriate messages
Algorithms	Appropriate icons
Variables	Relevant data formats for display
Software design approach (structured, agile etc)	Ergonomic issues
Quality assurance	Relevance to the user's environment and computer configuration
System and data models	Social and Ethical Issues
Project management tools	
Documentation	

Fig 1.1.6

Evaluation throughout the development process is undertaken with reference to these specifications. Specifications relevant to the user's perspective can be evaluated by the user, whereas specifications relevant to the developer can be evaluated by the developer.

Non-functional specifications from the developer's perspective.

These specifications will create a standard framework under which the team of developers must work. These are specifications that will not directly affect the product from the user's perspective but will provide a framework in which the team of developers will operate. The methods used to model the system - for example, use of flowcharts, class diagrams, structure charts - will be specified.

The method and depth of algorithm description to be used, how data structures, data types and variable names are to be allocated and any naming conventions that should be used should all be specified. A system for maintaining an accurate data dictionary needs to be specified. In other words, a framework for the organisation of the development process is set up so that each member of the development team will be creating sections of the solution that look, feel and are documented using a common approach (and so that each module will operate correctly with other modules).

Often CASE tools will be utilised to ensure the team complies with these non-functional specifications. Once these specifications have been developed a system model can be created to give an overview of the entire system. The system model will lead to the allocation of specific tasks for completion by team members, whilst at the same time, the overall direction of the project can be visualised.

CLASS DISCUSSION

Discuss: Non-functional Specifications: Developer
What are each of the items listed under 'Developers perspective' in Fig 1.1.6?
Why is it important to specify details of each of these items?

Non-functional specifications from the user's perspective.

Specifications developed from the user's point of view should include any non-functional specifications that influence the experience of the end-user. Standards for interface (or screen) design will be specified to ensure continuity of design across the project's screens. For example, use of menus, use of frames to group items, use of colour, placement of common elements. The wording of messages, design of icons and the format of any data presented to the user need to be determined and a set of specifications created. For example, the font to be used for prompts and user inputs, the size and nature of icons used, the tone of the language used.

Ergonomic issues should also be considered as part of the design specifications. Questions such as: Which functions will be accessed most often? Which functions require keyboard shortcuts? How should movement between screen elements be accomplished? What is the order in which data will be entered? Are the screens aesthetically pleasing? Such questions need to be examined and a standard set of non-functional specifications generated.

The user's environment will also have an effect on the specifications created. Perhaps the users are familiar with existing applications. If so, these applications should be examined and some of their design elements incorporated so that transfer of skills from existing applications can take place. For example, using keyboard short-cuts that are already known from other applications. Operating system settings should be used to determine the following: colour of windows, typefaces used and printer settings.

Consideration of hardware necessary to execute the new product including: processor speed, RAM, video memory and hard disk space. Potential users need to be consulted to determine the acceptable specifications.

System models can assist in determining user-based non-functional specifications, in particular, screen designs and concept prototypes. It is vital that software developers acknowledge the contribution users make to the development of design specifications. Communication and in particular feedback from users is especially important at this early stage.

CLASS DISCUSSION

Discuss: Non-functional Specifications: User
Consider an application installed on your calss computers. Discuss each of the items under "User's Perspective" in Fig 1.1.6 that have been considered and addressed in this application.

Discuss: Gathering User Specifications
Concept prototypes are one method for gathering user specifications. Brainstorm and discuss other possible strategies that could be used to gather user specifications.

EXERCISE 1.1.1 - MULTIPLE CHOICE

1. Which of the following is true for the Structured Development Approach?

   A. Suited to small projects, with clear requirements and performance is not considered to be critical.
   B. Suited to large projects, with evolving requirements and performance is critical.
   C. Suited to large projects, with clearly defined requirements and performance is critical.
   D. Suited to large projects, with evolving requirements and performance is not considered to be critical.

2. Which of the following is true for the Agile Development Approach?

   A. Suited to user-centred projects, with clear requirements and performance is not considered to be critical.
   B. Suited to user-centred projects, with evolving requirements and performance is not considered to be critical.
   C. Suited to backend machine-centred projects, with clear requirements and performance is critical.
   D. Suited to backend machine-centred projects, with evolving requirements and performance is not considered to be critical.

3. The correct order of stages when developing software using the Waterfall Development Approach is:
   A. Requirements definition, determining specifications, design, development, integration, testing and debugging, installation, maintenance.
   B. Determining specifications, requirements definition, design, development, integration, testing and debugging, installation, maintenance.
   C. Requirements definition, determining specifications, development, design, integration, testing and debugging, installation, maintenance.
   D. Requirements definition, design, determining specifications, development, testing and debugging, integration, installation, maintenance.

4. Which of the following words is NOT indicative of Agile Development?

   A. Agility
   B. Collaboration
   C. Flexibility
   D. Sequential

5. When a team is enrolled to work on a software project, which expert is likely responsible for formulating the set of requirements?

   A. Software Engineer
   B. Business Systems Analyst
   C. Programmer
   D. Users

6. In terms of Requirements Definition, what does it mean for a requirement to be verifiable?

   A. Requirements are worded in such a way that it is straightforward for both the client to accept and the developer to demonstrate that the requirement has indeed been fulfilled.
   B. The requirement should be stated in such a away that it includes full details with regard to how the requirement will be met as the software is designed and developed.
   C. All requirements should focus on a user need and they should be expressed in the form "As [role], I want [goal], so that [benefit/value]".
   D. Verifiable requirements must be expressed mathematically so that when tested there is a clear boundary between what is acceptable and what is unacceptable.

7. Watching users and recording the nature of the nature of tasks they perform is likely part of which of the following?

   A. Surveys
   B. Interviews
   C. Time management studies
   D. Business analysis

8. Which of the following best describes the purpose of user stories?

   A. To provide evidence to management that supports complaints (and compliments) from users of an existing system.
   B. To ensure user's needs are considered as requirements are developed and refined.
   C. To maximise the usability and effectiveness of a software product for its intended users.
   D. To capture and communicate specific functionality from users in order to guide and prioritise development efforts

9. Which of the following best describes the relationship between a system the environment in which it operates?

   A. The system is part or subset of the environment, and both system and environment exist within a boundary. The boundary prevents communication except via an interface.
   B. A system is confined to what it can control, or what is within scope by its boundaries. Interfaces enable the system to communicate with other systems or entities within the environment.
   C. The boundary of a system includes all the entities that provide input or receive output from the system. The system's environment operates outside the system and cannot effect the system except via an interface.
   D. An interface defines the limits of the system – what is an what is not part of the system. Boundaries are the mechanism used by the system to obtain input and send output to other systems present in the environment.

10. Which statement best distinguishes between functional and non-functional specifications?

    A. Functional specifications describe processes the software must perform, whilst non-functional specifications focus on how the software will be developed and how it will operate from the user's perspective.
    B. Functional specifications focus on how the software will be developed and how it will operate from the user's perspective, whilst non-functional specifications describe processes the software must perform.
    C. Functional specifications are a refinement of the requirements into needs expressed by the users, whilst non-functional specifications are items that ensure the experience of users is ergonomically, socially and ethically sound.
    D. Functional specifications are items that ensure the experience of users is ergonomically, socially and ethically sound, whilst non-functional specifications are a refinement of the requirements into needs expressed by the users.

EXERCISE 1.1.1 – SHORT ANSWER

11. Historically, the Structured or Waterfall Development Approach was the recommended method for developing software. Describe reasons why this is no longer the case.

12. Compare and contrast the Agile Approach with the Structured Development Approach.

Refer to the following scenarios when answering Questions 13, 14 and 15.

I. A new dam requires a software package to control the operation of its floodgates.
II. The water board wishes to computerise its meter reading activities.
III. A small business has a unique distribution and invoicing system.
IV. A chain of takeaway food outlets has grown and now wishes to build a phone app to provide a more personalised ordering experience to its customers.
V. A mail order company developing a web site allowing customers to order and pay online.
VI. A security company developing a computer-controlled alarm system.

13. Identify the personnel who should be considered/canvassed when defining the requirements for each of the above systems.

14. Do you think a custom designed solution is required for each of the examples above? Justify your response in each case.

15. Brainstorm at least 3 functional and at least 5 non-functional specifications that would likely be required for each of the scenarios.

System and Data Modelling Tools

The course specifications for Software Engineering published by the NSW Education Standards Authority (NESA) includes the following modelling tools:

- Context diagrams (Level 0 Data flow diagrams)
- Data flow diagrams
- Structure charts
- Data dictionary
- Storyboard
- Decision trees (and decision tables)

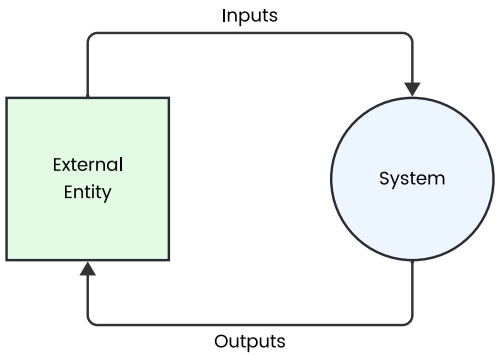
Note that Class diagrams are also included in the Software Engineering course specifications. Class diagrams are specific to the object-oriented programming paradigm and hence are introduced as required. The other modelling tools will be used throughout the course. What follows is an introduction to each tool along with an example. You will become more familiar with each tool, including creating examples as you design and develop your own software.

A model of a system is a representation of that system designed to show the structure and functionality of the system. Many system modelling tools are in the form of diagrams. Diagrams are particularly useful methods of modelling because they are able to give a broad view whilst at the same time conveying necessary detail.

Accomplishing the same task in words is difficult. In terms of software development, a model can be thought of as a plan. The model gives direction and specifications to the builders of the product, in the same way as the plan for a house gives builders of the house direction and specification. Different types of modelling are applicable to different aspects of the system. System flow charts are used to represent the logic and movement of data between the system's components, including hardware, software and manual components. Data flow diagrams describe the flow of data to and from processes and storage elements. Structure charts describe the top-down design and sequence of processing. Storyboards describe the navigation between screens. Data dictionaries describe the nature and type of the data used in a program. Screen designs and concept prototypes are used to determine user requirements by simulating the final product from the user's perspective.
Most projects will use a combination of modelling techniques.

Context Diagrams or Level 0 Data flow Diagram

Context diagrams represent the entire system as a single process. They do not attempt to describe the processes within the system; rather they identify the data entering and the information leaving the system together with its source and its destination (sink). The sources and sinks are called "external entities". As is implied by the word external, these entities are present within the system's environment. Context diagrams are really top-level data flow diagrams and are often known as level 0 data flow diagrams.



Data flows between the system and external entities

Fig 1.1.7

Squares are used to represent each of the external entities. Common examples of external entities include users, other organisations and other systems. These entities are not part of the system being described, as they do not perform any of the system's processes. Rather the system acquires (inputs) data from each source entity and/or the system supplies (outputs) information to each sink entity. The entire system is represented using a circle, with labelled data flow arrows used to describe the data and its direction of flow between the system and its external entities. Data flows from each source into the system, and data (information) flows from the system to each sink. In some references, external entities are also known as terminators as they describe starting and ending points for the DFD.

Each data flow label should clearly identify the nature of the data using simple clear words. Remember each data flow describes data not a process, for example if a user enters a password then an appropriate data flow label would be "User password", not "Enter password". Furthermore in this example each user is the source of a single password, so "User password" is a more appropriate label than "User passwords". If many data items flow together then a plural label would be more appropriate, however in most systems this is a rare occurrence.

All data entering the system (inputs)and all data leaving the system (information/outputs)must be included on the context diagram. All processes performed by the system and data stored by the system are part of the single system circle and are not detailed on the context diagram.

Consider the context diagram in Fig 1.1.8. One of the sales team enters a list of required products and a date range and the system responds by outputting a widget sales graph. Clearly this system is storing sales data and is performing a variety of processes using this data to generate the sales graph. Fig 1.1.9 shows an alternate version of the Widget sales analysis context diagram to illustrate how external entities can be included multiple times. Both versions are logically identical.

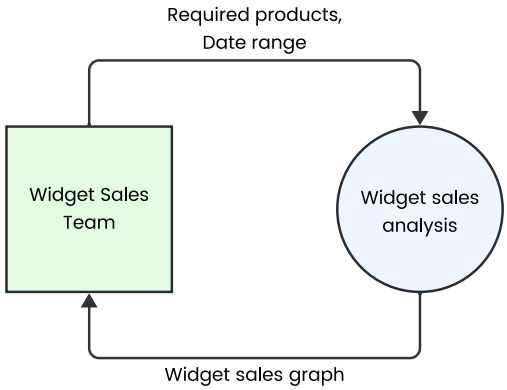


Fig 1.1.8

CONTEXT DIAGRAM OR LEVEL 0 DFD

Fig 1.1.9

So how does a context diagram assist when understanding the problem to refine specifications? Context diagrams indicate where the new system interfaces with its environment. They define the data and information that passes through each interface and in which direction it travels. Descriptions of this data and information is further detailed within a data dictionary. Ultimately the data entering the system from all its sources must be sufficient to create all the information leaving the system to its sinks.

Problem: Context Diagrams
A software application uses search criteria entered by the user and then creates and sends a query request to an external database server. The database server responds with the query results. The software application then formats the query results and displays the formatted results on the user's screen.

Construct a context diagram for this system.

Data flow Diagrams (DFDs)

Data flow diagrams describe the path data takes through a system. No attempt is made to indicate the timing of events. Think of a DFD as a railroad map: it shows where the train tracks are laid, but it does not give the timetables. DFDs do not attempt to describe the step-by-step logic of individual processes within a system. Rather they describe the movement and changes in data between processes. As all processes alter data then the data leaving or output from a process must be different in some way to the data that entered or was input into that process. This is what all processes do; they alter data in some way.

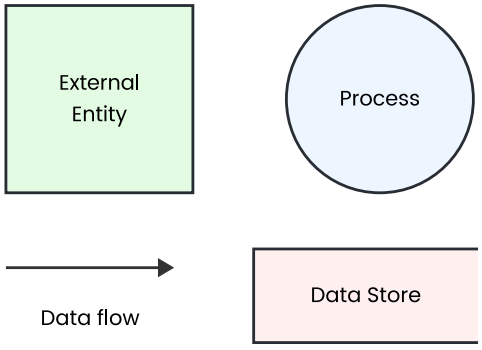


Fig 1.1.10

The aim of DFDs is to represent systems by describing the changes in data as it passes through processes. For example a process that adds up numbers receives various numbers as its input and outputs their sum. On DFDs there is no attempt to describe how the numbers are summed. Rather the emphasis is on where the numbers come from and where the sum is headed.

To represent the data moving between processes we use labelled data flow arrows. The label describes the data and the direction of the arrow describes the movement. Processes are represented using circles. The label within the circle describes the process. As processes change data the labels used should imply some action – verbs should be used, such as create, update, collect, to emphasise that some action is performed.

The final symbol used on DFDs represents data stores. A data store is where data is maintained prior to and after it has been processed. In most cases a data store will be a file or database stored on a secondary storage device, however it could also be some form of non-computer storage such as a paper file within a filing cabinet. An open rectangle together with a descriptive label is used to represent data stores. Data stores allow the system to pause or halt between processes and they also allow processes to occur in different sequences and at different times. In effect processes are freed to execute independently of each other. Consider a typical process that collects data from a user and stores it within a data store. This single process can execute many times simultaneously whilst at other times it sits idle. The data is maintained within the data store where it can be retrieved and used by other processes when and as they require.

As data stores are simply collections of data they cannot perform processing, therefore all data flow arrows heading into and out of a data store must be connected to a process. A process is required to write or read data to and from a data store. A level 1 data flow diagram expands the single process (or system) from a context diagram into multiple processes. A series of Level 2 DFDs are drawn to expand each level 1 DFD process into further processes. Level 3 DFDs similarly expand each level 2 process and so on. A series of progressively more and more detailed DFDs refine the system into its component sub-processes. Eventually the lowest level DFDs will contain processes that can be solved independently. Breaking down a system's processes into smaller and smaller sub-processes is known as 'top-down design'.

The component sub-processes can be solved and even tested independent of other processes. Once all the sub-processes are solved and working as expected they combine to form the complete solution. On some level 1 and lower-level DFDs the external entities are included, whilst on others they are not. If a context diagram has already been produced or external entities have been included on a higher-level DFD then it is common practice to omit the external entities from the derived lower-level DFDs. A similar practice is also true for data stores, however in the interest of improved clarity it is more common to reproduce data stores on lower-level DFDs. As was the case with context diagrams, to improve clarity it is permissible to include the same external entity multiple times and similarly a single data store can be included multiple times. For instance in Fig 1.5 below the "Widget Sales Team" entity is included twice simply to improve readability.

LEVEL 1 DFD

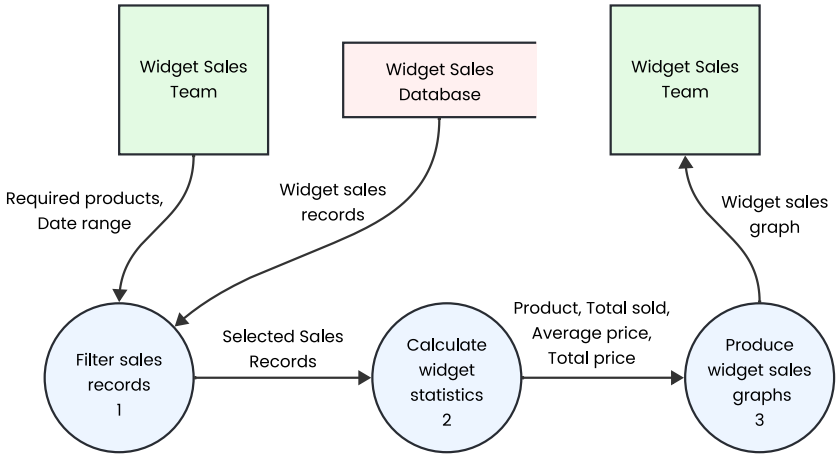


Fig 1.1.11

On most DFDs the processes are numbered in addition to their labels. Consider the example level 1 DFD in Fig 1.1.11 – it contains the three processes:

1. Filter sales records
2. Calculate widget statistics and
3. Produce widget sales graphs.

Three level 2 DFDs would then be produced – one for each process in the level 1 DFD. Fig 1.1.12 shows an expansion of process 2. Calculate widget statistics into a level 2 DFD containing four processes.

These four processes are numbered from 2.1 to 2.4 – the 2 indicating their connection to process 2 on the level 1 DFD. If process 2.1 required further expansion into a level 3 DFD then its processes would be numbered 2.1.1, 2.1.2, 2.1.3 and so on. Data flow diagrams are used both as a tool for system analysis when understanding the problem and also as a tool for system design. Modelling an existing system using a data flow diagram assists in understanding the flow of information through a system and also helps the analyst to separate processing into distinct units resulting in a better understanding of the problem.

CALCULATE WIDGET STATISTICS:

LEVEL 2 DFD

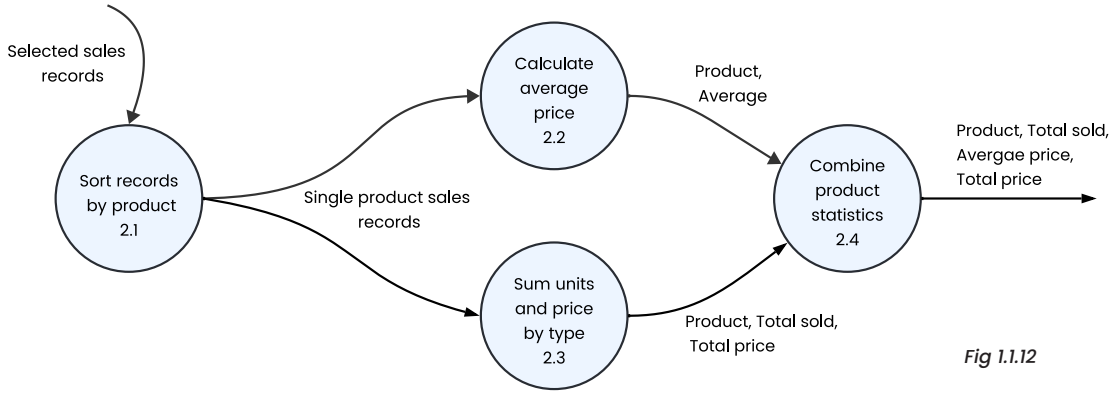


Fig 1.1.12

CLASS DISCUSSION & ACTIVITY

Discuss: Level 1 and 2 DFDs
Based on the above Widget sales analysis system modelled in Fig 1.1.9, 1.1.11 and 1.1.12, discuss a suitable format for the sales graphs.

Activity: Widget DFD
Identify examples within Fig 1.1.9, 1.1.11 and 1.1.12 that illustrate each of the above dot points.

Consider the following DFD summary points:

- All processes must have a different set of inputs and outputs.
- All lower-level DFDs must have identical inputs and outputs as the higher-level process they expand.
- External entities and data stores can be reproduced on lower-level DFDs.
- External entities must be present on context diagrams (level 0 DFDs) but are optional on lower-level DFDs.
- Data flows into and out of a data store must be connected to a process. They can never directly connect to an external entity or another data store.
- A single output data flow can be the input to multiple other processes.
- Labels for processes should include verbs that describe the action taking place.

HSC STYLE QUESTION

Problem: Conference Admission System DFD
An automated conference admission system has been designed to allow for the efficient registration of conference attendees as they arrive at the conference venue. The system operates according to the following guidelines:

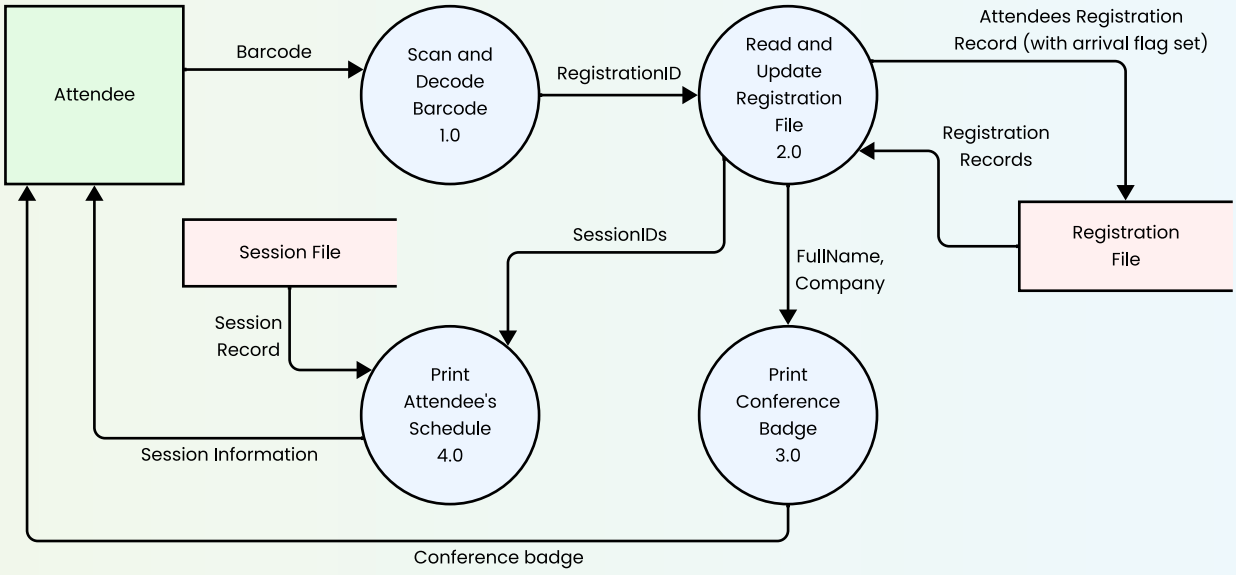
- Upon arrival at the conference centre the attendee scans the barcode on a printout (which they printed as part of their online conference registration). The barcode encodes the attendees unique RegistrationID.

- The system updates the existing Registration File to indicate that the attendee has arrived. This file contains a Registration record for each attendee, including their RegistrationID, Fullname, Company, Arrival flag and the SessionID for each session they are scheduled to attend.
- The system then prints the attendee's personalised conference badge, which includes their FullName and Company from the Registration File.
- The system also prints information about each of the attendee's scheduled sessions. The existing Session File contains a Session Record for each session. Each record contains SessionID, Time, Title, Presenter and Description.

A context diagram for the above system has been prepared.

Construct a data flow diagram for the Conference Admission System

Suggested solution



Note: *The detail of the Registration Records being read from the file. The correct record is found and used three times, namely to update the record and also to provide FullName and Company to the Print conference badge process and SessionIDs to the Print attendee's schedule process.*

Structure Charts

Structure charts (or structure diagrams) are used to model the hierarchy of subroutines (processes) within a system, together with the sequence in which these subroutines are executed. Data movements between subroutines are included to improve the reader's understanding of the relationships between them. In terms of software development, structure charts describe the top-down design of the program together with the order in which subroutines are called. Because subroutines can be conditionally called or repeatedly called from a higher-level subroutine, decisions and repetitions are also indicated on structure charts.

Key Term: Call
Causing the execution of a subroutine (process) from within another subroutine.

The primary function of a structure chart is to create a template in preparation for the creation of the actual source code. Each subroutine shown on the structure chart will become a procedure or function in the final program. For large software projects, structure charts may be used to allocate tasks to individual programmers on the development team. Structure charts are also useful tools for maintaining and upgrading software. Identifying subroutines and modules that could potentially contain an error or that need upgrading is made easier if a structure chart is available. Software tools are available that can analyse existing source code and automatically generate models similar to structure charts.

There are many recognised techniques for drawing structure chart type models and many different names are used for these diagrams. Some common names include structure diagram, function chart, call graph, call tree and hierarchy diagram. The important common theme for all these modelling techniques is that they attempt to describe the top-down, hierarchical design of a system. Some include an indication of the sequence of execution and methods for indicating decisions and repetitions.

Key Term: Control
The influence that causes tasks to execute in their correct order

Let us now consider the symbols used in structure charts as they are to be drawn in this course. Processes, which are usually subroutines and sometimes modules, are drawn as rectangles. Lines drawn between processes are known as call lines or control lines. They show the connections between processes. Each line represents a call to a subroutine from the subroutine above. Decisions about whether a call should be made are indicated using a small diamond. If a subroutine is to be called multiple times, then a circular arrow to indicate this repetition is placed around the call line. Data items passed between tasks are known as parameters and are shown using a small open circle with an arrow attached to indicate the direction of the data movement along the call line. A filled in circle with attached arrow is used to show a control parameter. Control parameters have an effect on the order in which tasks are executed. Usually, a control parameter will be a flag. Flags are data items used to indicate that a certain criterion has been met.

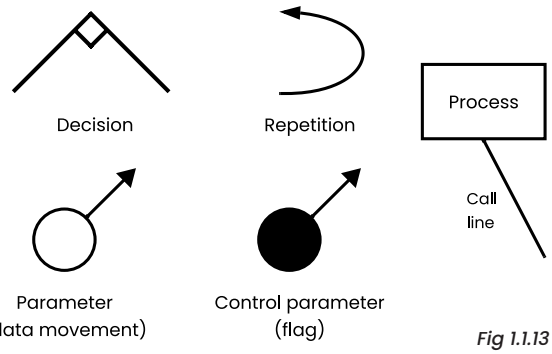


Fig 1.1.13

Structure charts can be read from top to bottom. Lower-levels represent subroutines that are called from the subroutine above. Reading from left to right on a particular level describes the execution order of tasks. Sometimes a subroutine may be called by more than one higher-level subroutine, because order of execution is from left to right. It is necessary to include this subroutine each time it is called by a different higher-level task.

CLASS ACTIVITY

Structure Chart & Algorithms: Automotive Parts Business

A program is being developed for an automotive spare parts business. The structure chart in Fig 1.7 below describes the subroutines used to determine the number of items currently held in stock for particular parts.

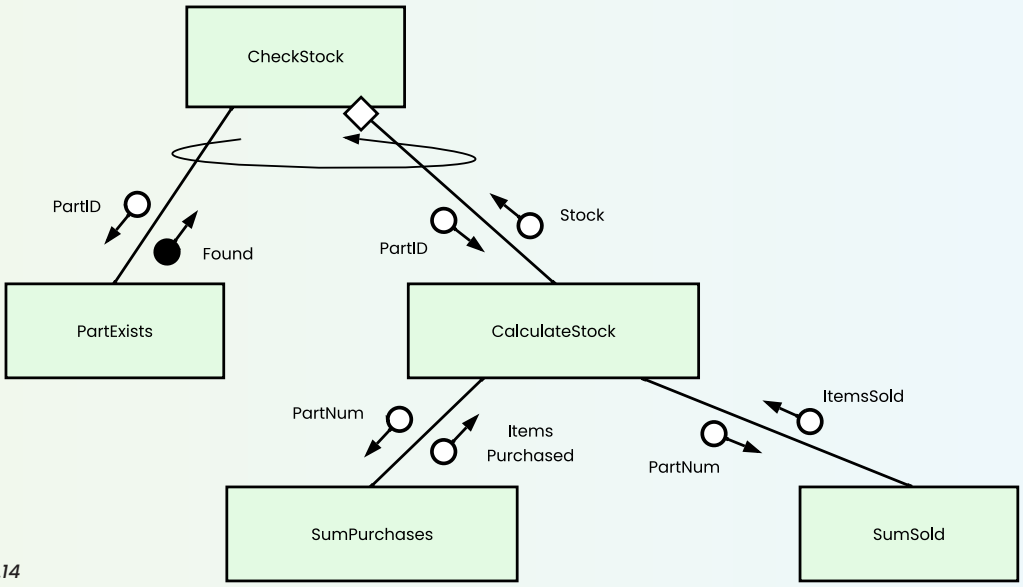


Fig 1.1.14

The following algorithms (page 30) have been written based on the structure chart in Fig 1.1.14 above. Algorithms for the `SumPurchases` and `SumSold` subroutines are yet to be developed. Line numbers are included to assist with the discussion that follows.

Algorithm: Automotive Parts Business

```
1     BEGIN CheckStock
2           Get PartID
3           WHILE PartID ≠ 0
4                 IF PartExists(PartID) THEN
5                       Stock = CalculateStock(PartID)
6                       Display Stock " item(s) in stock"
7                 ELSE
8                       Display "Invalid part number."
9                 ENDIF
10                Get PartID
11          ENDWHILE
12    END CheckStock

13    BEGIN PartExists(PartNum)
14          Found = False
15          Open Parts file for input
16          Read first record into TempRec from Parts
17          WHILE TempRec.PartID ≠ 0 AND Found = False
18                IF TempRec.PartID = PartNum THEN
19                      Found = True
20                ELSE
21                      Read next record into TempRec from Parts
22                ENDIF
23          ENDWHILE
24          Close Parts file
25          RETURN Found
26    END PartExists

27    BEGIN CalculateStock(PartNum)
28          ItemsPurchased = SumPuchases(PartNum)
29          ItemsSold = SumSold(PartNum)
30          Stock = ItemsPurchased - ItemsSold
31          RETURN Stock
32    END CalculateStock
```

First consider how the structure chart represents the hierarchy or top-down design of the subroutines. In the structure chart we have a total of five subroutines. The `CheckStock` subroutine calls both the `PartExists` and the `CalculateStock` subroutines. The `CalculateStock` subroutine calls the `SumPurchases` and the SumSold subroutines. Notice how the top-down design is very obvious on the structure chart but is somewhat difficult to determine using the algorithms alone.

The left to right placement of the subroutines within the structure chart indicates the order in which they are called. For example, the `CheckStock` subroutine first calls `PartExists` then calls `CalculateStock`, similarly the `CalculateStock` subroutine calls SumPurchases and then calls `SumSold`.

On the structure chart calls from the CheckStock subroutine are surrounded by a repetition symbol. This means that within the CheckStock algorithm the two calls are made within a loop. Examining the algorithm we find the pre-test loop beginning at line 3 and ending at line 11 surrounds the PartExists and CalculateStock calls. A decision diamond ◇ is included on the CalculateStock call line. This means the CalculateStock call is optional based on a decision within the CheckStock algorithm. Examining the CheckStock algorithm we find line 4 to 9 includes the relevant logic which avoids calling CalculateStock if the part number is not valid.

Finally consider the parameters on the structure chart. Based solely on the structure chart it is clear that the subroutine PartExists receives a PartID and returns Found. The Found parameter is represented as a control parameter ♂. This implies it is most likely a Boolean which is used to make a decision on a future call. In this case Found is used to decide if the CalculateStock subroutine needs to be called. The CalculateStock subroutine receives a PartID and returns the Stock of that part. The SumPurchases subroutine is sent a PartNum and returns the number of ItemsPurchased, similarly SumSold receives a PartNum and returns the number of ItemsSold. As these parameters use the symbol ⚷ they are not control parameters and hence they do not alter the order of processing (or flow of control) within the algorithms. Rather these parameters are used to pass data into and out of subroutines.

CLASS DISCUSSION

Discuss: Structure Charts
Structure charts can be created from existing algorithms or even from existing source code. Examine the algorithms above to determine how they convert to the initial structure chart.

Discuss: PartID
The PartID entered by the user is utilised by all the subroutines. Describe how the PartID is passed to each of the subroutines.

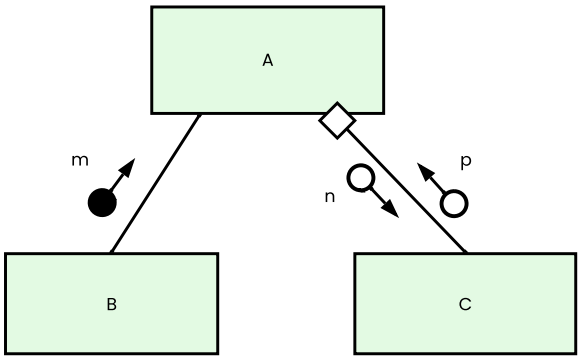
HSC STYLE QUESTION

Problem: Update High Scores
The following algorithm updates the high score data at the conclusion of a game. Create a structure chart for the UpdateHighScores algorithm.

```
BEGIN UpdateHighScores(PlayerName,Score)
      LoadHighScores
      PlayerIndex = FindPlayer(PlayerName)
      IF PlayerIndex < 0 THEN
            InsertPlayer(PlayerName, Score)
      ELSE
            IF Score > Player(PlayerIndex).HighScore THEN
                  UpdateScore(PlayerIndex, Score)
            ENDIF
      ENDIF
      WriteHighScores
END UpdateHighScores
```

HSC STYLE MULTIPLE CHOICE QUESTIONS

Consider the following structure chart when answering Question 1, 2 and 3.



1. Which of the following correctly identifies elements on the structure chart?

A. **m**, **n** and **p** are processes, **A**, **B** and **C** are parameters.
B. **n** and **p** are control parameters, **m** is a parameter, **A**, **B** and **C** are processes.
C. **m** is a parameter, **n** and **p** are data items, **A**, **B** and **C** are variables.
D. **m** is a control parameter, **n** and **p** are parameters, **A**, **B** and **C** are processes.

2. Which of the following best describes the structure chart?

A. **A** calls **B** which returns a Boolean value for **m**. **m** is used to decide if **C** is called. If **C** is called it returns **p** after processing the value **n**.
B. **B** calls **A** with the Boolean value **m**. **m** is used to decide if **C** should call **A**. If **A** is called it returns **n** after processing the value **p**.
C. **A** calls **B** which returns a Boolean value for **m**. **C** is called repeatedly using **n** values. Each time **C** is called it returns a value for **p**.
D. **A** calls **B** with a value for the parameter **m**. **C** is optionally called with the parameter **n** and returns a value for **p**.

3. Which of the following algorithms for A best reflects the above structure chart?

A.
```
BEGIN A
    B = m
    IF m THEN
        Get p
        C = B(n)
    END IF
END A
```

B.
```
BEGIN A
    B(m)
    C(n, p)
END A
```

C.
```
BEGIN A
    m = B
    IF m THEN
        Get n
        p = C(n)
    ENDIF
END A
```

D.
```
BEGIN A
    B(m)
    IF m THEN
        C(n, p)
    END IF
END A
```

Suggested solutions: 1D, 2A, 3C

Structure Chart: Processing Invoices
A business is invoiced for supplies from its vendors. The structure chart below models the tasks involved in processing and paying these invoices.

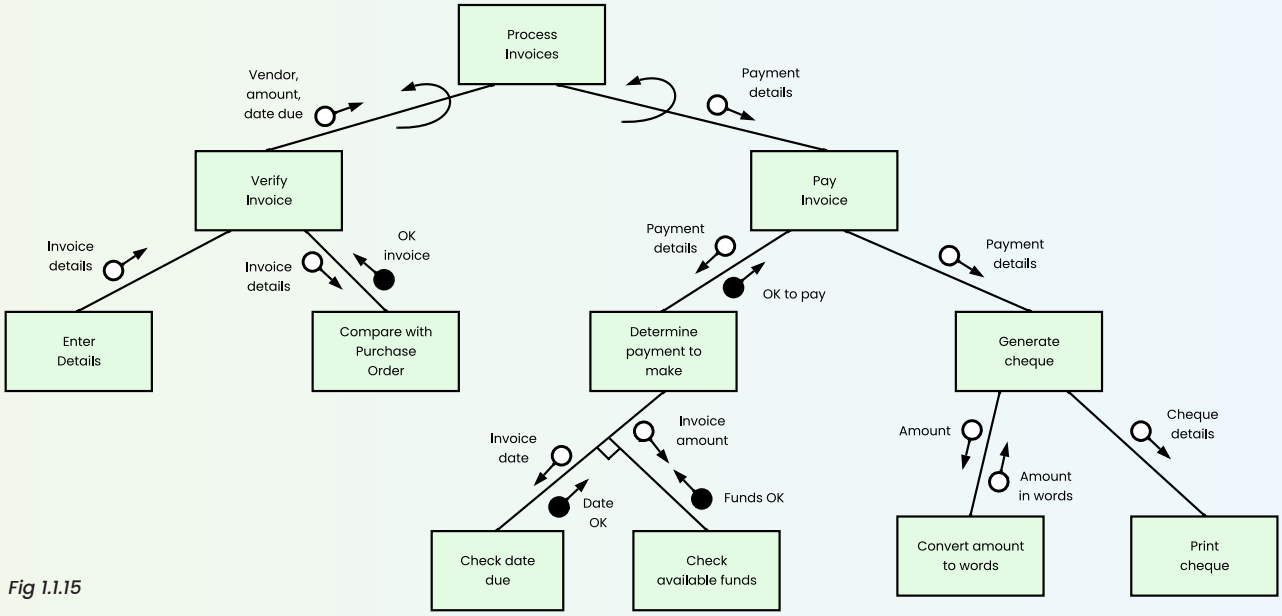


Fig 1.1.15

Discuss: Processing Invoices
Walk through the processing modelled on the above structure chart (Fig 1.8).
Consider:

1. The top-down design
2. The order of processing
3. The movement of data through the system

Discuss: Structure & Code
Discuss the link between structure diagrams and code. Use the above structure chart as an example during your discussion

Data Dictionary

When developing a software solution to a problem, it is vital that all variable names or identifiers are carefully documented. This ensures that all members of the development team are aware of identifiers that have been used, their data type, storage size, display size and format, scope of usage and various other details. A data dictionary is the repository where this information is held. Data dictionary functions are available with most software development CASE tools. If the data dictionary is an integral part of the development environment then confusion due to duplicate identifiers can be eliminated. For smaller development projects, the data dictionary is often maintained on paper with the assistance of a database or word processor.

A data dictionary should include a thorough description of every variable and data structure used by the program and each field in each database and file used or accessed by the program.

A separate data dictionary is commonly created for each module, database and file used by the program. Often a data dictionary will be a table containing columns for:

Key Term: Scope
The extent to which a variable is available for use. Global variables are available for use by all subroutines in a program. Local variables are available only within the individual subroutine

- **Variable** name or data item. This is the name used to access the variable within the source code.
- **Data type**, such as integer, string floating point or Boolean. For data structures the general nature of the structure is used, for example Array(string) specifies an array which contains a sequence of strings. When specifying records use a heading row followed by a separate row for each field within the record.
- **Format**. Refers to the display format where this is applicable, for example XXNNN for a string indicates a total of 5 characters where the first two characters are alphanumerics and the last three characters must be digits, such as BF345.
- **Number of bytes** required for storage. For strings each character will use a single byte if ASCII is used and two bytes if Unicode is used. For example, a string containing 5 characters will require either 5 or 10 bytes of storage.
- **Size for display**. The number of characters required to display the contents of the variable on a screen or on paper. This value will include decimal points, dollar signs, percentage signs or any appropriate characters added to the raw data when it is displayed. For instance, dollar and cents values less than 10,000 will require up to 8 characters to display as $9999.99 contains 8 characters. However, if the data type is single precision floating point then just 4 bytes are required for storage.
- **Description** or purpose. A brief outline of the logical role the variable plays within the subroutine. May include other relevant details such as whether it is a parameter, input by the user or stored in a file.
- **Example**. A typical example of an actual data item usually in its display format. This is particularly important for dates, currency and other formats that can be shown in a variety of different ways.
- **Validation**. Refers to the range of allowable values. For instance, a person's age in years must be positive and less than ~130. Dates are often restricted based on the current date, that is often they must be less than or equal to today's date or for delivery dates they must be after today's date.
- **Scope** of the variable. Is the variable local to its subroutine or is it available more widely?

The exact columns used will be determined by the nature of the data and how it is used in the source code. A variable that is only used internally (never displayed to users) has no need for a format and size for display, hence these can be left blank. An example of a data dictionary can be found in Fig 1.9.

The data dictionary below describes all the identifiers used in the Convert Amount to Words subroutine from the Invoicing System structure diagram shown in Fig 1.1.15.

Variable	Data Type	Number of bytes	Description	Example
AmountInWords	String	255	Returns the currency amount in words.	Six thousand, two hundred and fifty dollars and 45 cents.
Amount	Float	4	Input parameter	6250.45
TempDigit	Integer	2	Stores each digit as it is extracted from Amount	6
DigitWord	Array[String(19)]	10 * 20	The word associated with each digit.	DigitWord(5)="five".
TenMultipleWord	Array[String(9)]	10 * 10	Word for each multiple of ten.	TenMultipleWord(3)="thirty"
Ten3Word	Array[String(4)]	10 * 5	Word for each 3rd power of ten.	Ten3Word(2)="million"
PlaceCounter	Integer	2	Counter incremented for each digit in Amount.	1
TempResult	String	255	Stores the amount in words during processing.	and 45 cents.

Fig 1.1.16

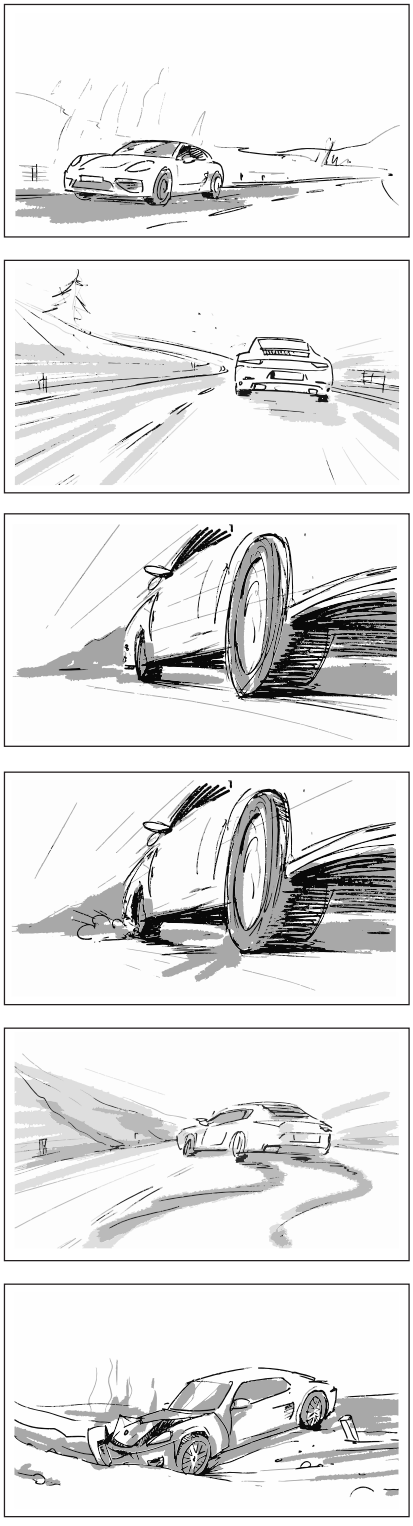
The programmer created this data dictionary whilst developing the source code. For this particular subroutine columns for format and size for display have not been included as the only output is the final amount in words, which is a string up to 255 characters long. Data dictionaries are stored, along with other documentation, to assist in future maintenance and upgrading of the software product.

CLASS DISCUSSION

Discuss: Data Dictionary
How could a data dictionary, such as the one above, be of assistance to future maintenance personnel?

Discuss: Diagram Commonalities
Explain the relationship between structure charts, DFDs, and data dictionaries. Use the above data dictionary as an example to illustrate your answer.

Storyboards



Storyboards are used to visually outline and sequence user interactions, providing a framework for designing and communicating the flow of a user interface or application. Effectively, a storyboard for a software product is a collection of screen designs together with a diagram describing the links between the screens. Storyboards are commonly used in movie production where a series of cartoon-like screens are produced. For movie production, the sequence in which screens are encountered is always linear, as the movie needs to be watched from start to finish to make sense. This is not the case with most software. The interactive nature of software means that storyboards must carefully document the possible flow of data between screens.

Storyboards for software products usually include a series of screen designs together with a diagram illustrating the navigation paths between screens. This diagram represents each screen as a rectangle with arrows leading to and from other screens. It is important to develop a logical navigation system between screens so users do not become lost in a maze of screens. Navigation buttons are normally included on every screen in a consistent position to assist users.

Various methods exist for organizing navigation between screens in software development. The most common approach involves a hierarchical structure, where a main menu directs users to major screens, each with links to lower-level screens in the hierarchy.

The diagram below shows a hierarchical screen structure for a company's web site. Although the structure is basically hierarchical, a link is provided on each screen to return the user to the home page.

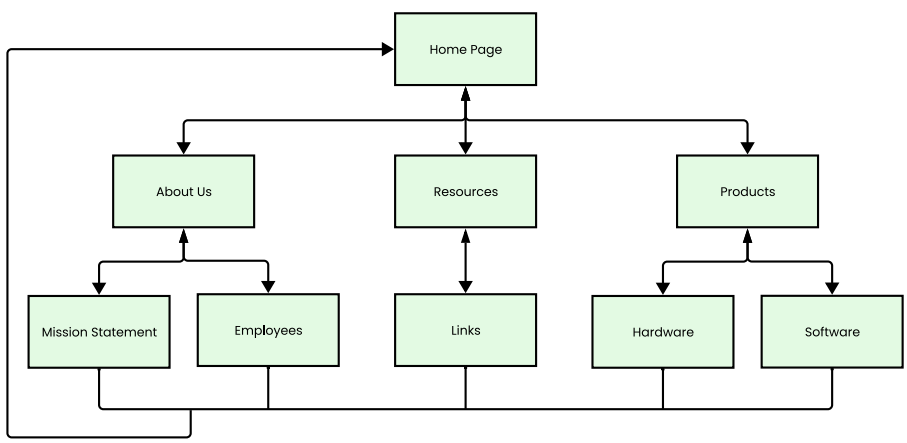


Fig 1.1.17

CLASS DISCUSSION

Discuss: Storyboards
In Fig 1.1.17 is there any way of viewing the Hardware page without first viewing the Products page? Suggest reasons why the developer may have done this.

CLASS DISCUSSION

Discuss: Screen Elements
Fig 1.1.18 presents a storyboard for an online shopping application. Note that unlike traditional storyboards that are often hand-drawn, these screens have been digitally created.

List and describe the screen elements shown in the designs below. Brainstorm some suggestions for improving this storyboard.

Activity: Storyboards
Initial screen designs/storyboards are often done on paper. Create a new and improved alternative storyboard for the online shopping application, including annotations.

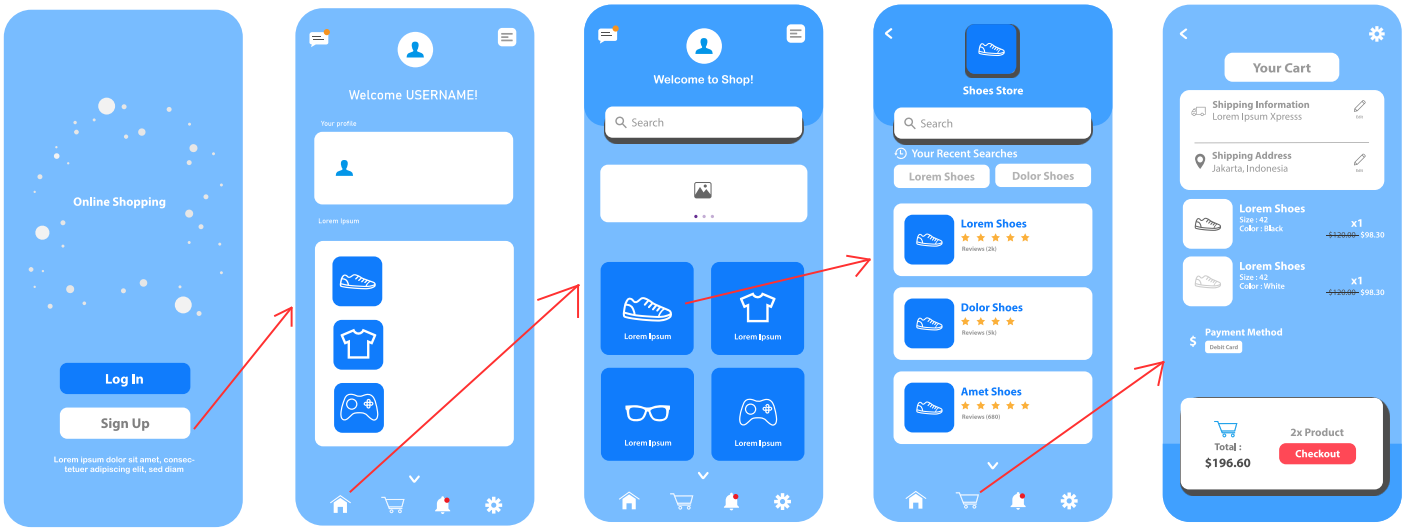


Fig 1.1.18

Project Management Tools

Project management tools are used to document and communicate:

- What each task is,
- Who completes each task,
- When each task is to be completed,
- How much time is available to complete each task, and
- How much money is available to complete each task.

Without such documentation and planning, time and budget overruns are likely, and problems leading to overruns are challenging to detect until it is too late. Lack of planning is a major reason for project failure; poor planning can even result in project abandonment. Project management documentation must acknowledge that challenges are inherent in virtually all projects, requiring flexibility to reassign tasks, allocate resources, adjust budgets, and manage timelines effectively.

Project managers use a variety of project management tools including:

- Gantt charts for scheduling of tasks.
- Journals and diaries for recording the completion of tasks and other details.
- Funding management plan for allocating money to tasks.
- Communication management plans to specify how all stakeholders will communicate with each other during the development of the new system.

Gantt charts for scheduling of tasks

A Gantt chart is a horizontal bar chart used to graphically schedule and track individual tasks within a project. The horizontal axis represents the total time for the project and is broken down into appropriate time intervals – days, weeks or even months. The vertical axis represents each of the project tasks. Horizontal bars of varying lengths show the sequence, timing and length of each task.

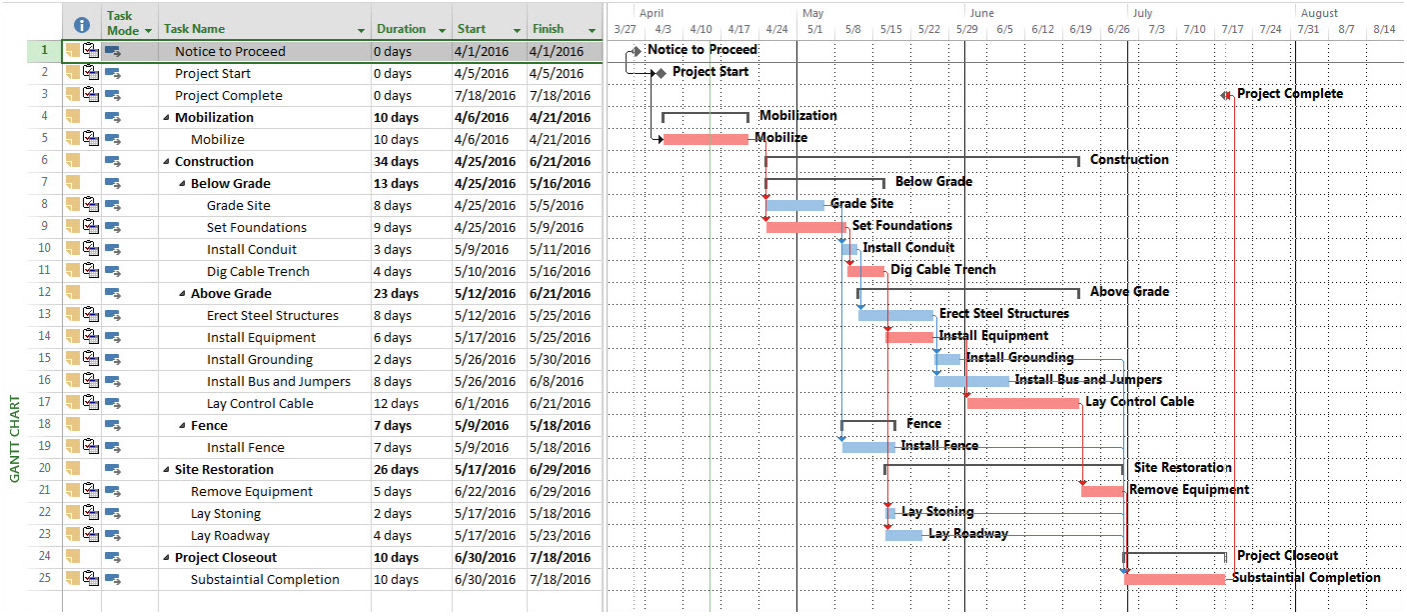


Fig 1.1.19

Fig 1.1.19 shows a sample Gantt chart. Project checkpoints or milestones should be planned to signify the completion of significant tasks. Milestones are particular points in time; they have no duration and do not require work. Rather they are flags indicating that work should or has been completed. During development, reaching each milestone is an indication to management that the project is progressing as intended. Milestones are also times when the overall progress is assessed, which may result in changes to the schedule or various other aspects of the project's management.

ACTIVITY

Activity: Gantt Charts
Create a Gantt chart to describe the sequence of stages for a project. Examples of projects could be building a house, purchasing a car, different licences from learners to full licence, creating an artwork, etc

Journals and diaries

Journals and diaries are tools for recording the day-to-day progress and detail of completed tasks. Diaries are arranged in chronological order with a page or section for each day's events. Meetings, appointments, tasks and any other events are recorded in advance. Both diaries and journals are used to record details of events that have recently occurred. After or during an event diaries tend to be used to record factual information, whilst journals include a more detailed analysis and reflection on recent events. However in terms of recording past events the distinction between the two is unclear.

Diaries are an organisational tool and a memory aid. Most individuals, teams and organisations maintain diaries. For example, your school administration probably has a diary where teachers record all future events that will affect other members of the school community, such as excursions, meetings and exam periods. The school diary is used by administration to generate the daily notices that are read out each morning or printed and distributed to staff. Teachers refer to the school diary prior to booking events to ensure they don't clash with other events. Project teams maintain diaries for similar reasons. For instance, the project manager records when meetings will occur and team members record appointments that will take them out of the office. Such information is critical if the team is to operate smoothly and effectively.

Journals, and sometimes diaries, document work completed by team members during the project's development. As tasks are completed team members write down what was done and any issues they encountered. In addition such comments can include ideas and comments on possible future improvements. Project managers refer to journals as they monitor the completion of tasks and identify possible issues. Through the use of journals, details of problems encountered and their effect on the development process can be analysed. New ideas and other comments can be discussed and considered when planning future projects.

CLASS DISCUSSION

Discuss: Diaries and Journals
Diaries and journals can be hand-written paper documents, however for many project teams networked software applications are used. Contrast paper-based journals and diaries with software-based journals and diaries.

Funding management plan

A funding management plan aims to ensure the project is developed within budget. For this to occur requires that each development task be allocated sufficient funds at the correct time and that these funds are spent wisely.

Funding management plans should specify:

- Allocation of funds to tasks. Will the funds be released in full before the task commences, progressively during the task or after the task is complete? Answers to such questions will depend on the individual nature of the task, the development approach being used and whether the task is completed in-house or is outsourced for completion by an external party.

- Mechanisms to ensure money is spent wisely throughout the SDLC. The plan should specify the procedures to be followed each time a product or service is ordered during the system's development. For example, often three quotations are required for all significant purchases and full payment is not to be made until after the product has been received and checked.

- Accountability for each task's budget. Ultimately the money spent on every detail of the system's development contributes to the total development process remaining within budget. Therefore someone should be accountable for ensuring each task is completed within its own slice of the total budget. Funding management plans should detail who this person is for each task, together with procedures they must follow to allow management to monitor their use of funds.

- Reallocating funds during development. Funding plans should include sufficient flexibility so that funds can be redirected should problems occur. Unforseen circumstances almost always occur, the funding plan should recognise and plan for such occurrences.

Communication management plan

It is vital that all parties involved in the system's development communicate with each other effectively. Communication management plans specify how this communication occurs. Strategies documented in the communication management plan provide a structure that supports effective ongoing communication between all team members throughout the project's development.

A typical communication management plan should specify:

- The communication medium to be used, for example e-mail, newsgroups, facsimile, meetings, weekly bulletins or even telephone calls. Different types of communication are likely to be effective under different circumstances. For instance facsimile may be specified for quotations whilst email is likely to be a more suitable medium for informal communication between software developers.

- The lines of communication. The communication management plan should specify how each party is able to obtain answers to questions or communicate other details to and from other project team members and the client. For example, it may well be appropriate for the systems analyst to contact the client directly, however it may not be appropriate for a programmer working on a specific part of the solution to do so. The communication management plan should specify the lines of communication the programmer must negotiate to obtain answers from the client.

- Methods for monitoring the progress of the system's development. This includes completion of tasks, monitoring costs and also verifying requirements as part of ongoing testing. For example, meetings can be scheduled to check critical tasks will be completed on time.

- Changing or emerging requirements. During the development of most projects new requirements will emerge or existing requirements will require alteration. The communication management plan should pre-empt such occurrences so that these new or changed requirements can be effectively communicated to all parties.

CLASS DISCUSSION

Discuss: Management Tools
List different methods or mediums of communication, such as email, weekly bulletins, meetings, etc. Discuss when each would be appropriate during a project's development.

EXERCISE 1.1.2 – MULTIPLE CHOICE

1. A model of a system can best be described as:

   A. a data flow diagram representing logic and movement of the data.
   B. a representation of a system showing structure and functionality.
   C. a data dictionary describing the data types used by a system.
   D. a structure chart showing the navigation between screens.

2. Storyboards identify:

   A. the data types used.
   B. the screen designs.
   C. how inputs are transformed into outputs.
   D. the flow of data.

3. Data flow diagrams explain:

   A. the data path through the system.
   B. the timing of the data path through the system.
   C. the manual processes component of the system.
   D. the sequence of processing.

4. Structure charts are used to:

   A. model the hierarchy of the processes within the system.
   B. show the algorithms that the system will use.
   C. determine user requirements.
   D. all of the above.

5. A data dictionary should include:

   A. identifiers.
   B. data types.
   C. number of bytes of storage.
   D. all of the above.

6. The interface between the end-user and the software is largely determined by:

   A. the operating system.
   B. the screen designs.
   C. the clients evaluating the system.
   D. the source code.

7. A collection of screen designs together with a diagram describing the links between screens is referred to as a(n):

   A. storyboard.
   B. IPO diagram.
   C. structure chart.
   D. hierarchical diagram.

8. Data dictionaries and other documentation should be stored because:

   A. it assists in the maintenance of the software.
   B. it assists in the upgrading of the software.
   C. it assists in understanding the processes used by the software.
   D. all of the above.

EXERCISE 1.1.2 – SHORT ANSWER

9. Structure charts have many different names. Compile a list of these and explain in your own words how each of these terms describes the nature of a structure chart.

10. Traditional board or card games have very definite rules. For this reason they are good examples to use when learning to apply the logic and structure that is involved in software development. Using this concept, draw a structure diagram for a board or card game with which you are familiar.

11. Data flow diagrams are particularly useful for describing systems that are data oriented. An EFTPOS (Electronic Funds Transfer Point Of Sale) terminal is used to process withdrawals directly from customers' accounts. These terminals are now commonplace in most retail stores.
    Design a data flow diagram to model the operations of a typical EFTPOS terminal.

Design

The design stage involves the creation of system models that will be used to identify and determine appropriate data structures and what algorithms will be needed prior to coding the solution. In the Year 11 course, we have a complete topic (and chapter) on each of these areas

- Designing algorithms
- Data for software engineering
- Developing solutions with code

In this section we briefly introduce each to give you a flavour for what is to come.

Designing Algorithms

An algorithm is a method of solving a problem. The algorithm describes the processing steps necessary to transform the inputs into the outputs. This occurs within a finite amount of time. Algorithms are used to assist in the solution of all types of problems not just computer-based problems. For example, a recipe is an algorithm describing the preparation and cooking steps required to create a meal. In most cases, recipe books call this the 'method'. The method is a sequence of steps that transform the ingredients (input) into the meal (output).

We use algorithms subconsciously. For example, each morning you go through a sequence of decisions and steps to prepare and travel to school. When searching for a lost article we try various techniques; we may try retracing our steps, looking in obvious places, asking other family members, etc... These are all valid methods of solving the problem; in essence you are using algorithms. The algorithm is not the solution, but the method used to arrive at the solution.

Bringing algorithms out of the subconscious and presenting them in such a way that others can understand them. This requires us to be able to describe the algorithms we devise, clearly and logically. There are various methods of algorithm description. In chapter 1.2 we will consider two - pseudocode and flowcharts.

CLASS DISCUSSION

Consider: Algorithms
Each morning most high school students need to make decisions about their day. The steps taking place may include the following:

1. Wake up
2. If you are sick then stay in bed.
3. If it is a school day then continue otherwise stay in bed.
4. Get out of bed.
5. Have a shower.
6. Get dressed in school uniform.
7. Eat breakfast.
8. Pack school bag.
9. If mum is home then scab a lift otherwise catch bus to school.

There are problems with the above steps if we work through them in the precise order indicated. If you are sick, then step 2 directs you to stay in bed. We then reach step three, which indicates that on school days we must complete the steps that follow. This seems to contradict step 2's direction. This algorithm has three exit points, at steps 2, 3 and 9. It would be better if we could restructure our algorithm to have a single exit.

Discussion:
Can you rewrite steps 1 to 9 from above in such a way that there is only one exit point from the algorithm? Ensure that your new algorithm performs the tasks in the manner intended in the original.

Task:
The above 9 steps do not correctly deal with all decision relevant to a typical school day, in fact, they merely get you to school. Extend the algorithm to work with other non-school days and to describe the return trip home

Data and Data Types

Data items are the raw materials on which computer programs operate. These data items must be stored in binary if they are to be manipulated by the instructions that form the software programs. Each data item must be assigned a data type. The data type determines how each of the data items will be represented in binary and what kind of processing the software will be able to perform on them.

There are a number of data types that are used so frequently that most programming languages include them as predefined parts of the language. These data types are those that are used in everyday life.

Data Type	Example data
Integer	-5, 0, 34, -9245, 89
Floating Point	-5.045, 2.3×108, 6.874
String	Fred, Yu3fk, Eggplant
Boolean	Yes, No, True, False
Date	16/1/23, 25 Jan 2023
Currency	$5.65, $19.05, -$3.70

Fig 1.1.20

For example, we use whole numbers (integers) for counting and performing arithmetic, numbers with decimal points or real numbers (floating point) for fractional and large number computations, and words and sentences (strings) for all forms of writing. We use yes/no or true/false (Boolean) data to answer questions and make decisions. Dates and times are used to schedule our lives and currency is used for purchasing. All these data types are predefined in most programming languages.

All data and instructions are stored in binary. In chapter 1.3 we examine the representation of numbers using the binary, decimal and hexadecimal systems. Then we will examine many of the data types listed in the above table. We investigate how they are represented in binary, together with any of their limitations.

CLASS DISCUSSION

Investigation: Calculation Errors
All calculators, calculator apps, spreadsheets, programming languages, etc. have numerical units. For example, on current iPhones the addition 100,000,000 + 0.9 in simple mode is 100,000,001. In scientific mode 10^{15} + 0.9 shows a similar precision error. Investigate the limits of calculators on your device. Consider large magnitude numbers - negative and positive - and also very small numbers close to 0. Can you explain why these errors occur?

Discussion: Data storage
Digital data is stored in binary. This includes text, numbers, graphics, video, audio and instructions. How has this assisted the development of devices that can utilise digital data from a variety of sources? Discuss.

Data Structures

Data structures are arrangements of data items; the aim being to simplify access and processing of the data. When designing data structures for particular problems, it is important to carefully consider the nature of the processing that will be required. Well-designed data structures can significantly reduce the complexity of the processing required. Conversely poor or inappropriate data structures can greatly increase the processing required and reduce the software's performance.

In chapter 1.3; "Data for Software Engineering", we will delve into the following data structures in detail:

- Arrays
- Lists
- Stacks
- Sequential files
- Records
- Trees
- Hash table

RESEARCH TASK

Research: Data Structures
There are many online videos that introduce data structures. After watching, write a sentence or two describing each of the above.

Development

Development is the stage where the solution is coded in a programming language. In the Software Engineering course, Python is specified as the language that will be used in the HSC examinations, therefore Python will be the focus of chapter 1.4; "Developing Solutions with Code". During this stage the design from the previous stage is put into practice. If a structured or waterfall approach is being used then the entire design is coded. If an agile approach is used, for example, SCRUM, then a part of the product is coded for the current iteration.

Multiple iterations (in the SCRUM version of agile, these are known as Sprints) are performed, progressively implementing new requirements until the product is deemed to be complete.

It is critical that code performs its functions correctly but it is also critical that it reacts appropriately to unexpected data and potential security breaches. Thorough debugging to test and evaluate the code is ongoing as the solution is coded.

There are various error detection and correction techniques with tools available within most modern integrated development environments (IDEs) to automate the debugging process. The testing process has become so critical to quality and secure code that Test Driven Development (TDD) has become popular. TDD requires specifications to be formed as a series of tests. Once the code is able to execute successfully for all tests then it is complete. Code is written to progressively pass the tests. Traditionally, tests were written to pass the code.

RESEARCH TASK

Research: Test Driven Development
Research Test Driven Development (TDD) and provide some examples of tests for a particular scenario.

Integration

Once code is created and initial testing is performed on the code, it may have to be integrated with other code that already exists such as existing modules or systems, including older legacy systems. Software may need to connect to external databases and API's. Other software applications may need to load Dynamic Link Libraries.

Several different people in an organisation may be working on different modules at the same time. This requires that the individual modules will need to be combined. Version Control Systems (VCS) track and manage changes to source code. They enable teams of programmers to work on a single project simultaneously with mechanisms in place to ensure the work of one developer doesn't overwrite that of another. In addition to improving the ability to integrate code from different developers, such software also provides backup of the code in such a way that any version of any module remains available should it be needed.

CLASS DISCUSSION

Discussion: JIRA
JIRA™ by Atlassian has an enormous market share of the commercial VCS market as does GitHub. Identify the main features available within these VCS products.

What is Git?

The term Git is a British slang word referring to a despicable unpleasant person. Linus Torvalds, the creator of Git, is thought to have named his software Git due to his frustration using existing version control systems (VCS). Git is currently the most widely used VCS and is highly respected by software developers.

Git is designed to manage source code and track changes in software development projects, in particular for collaborative coding of open-source projects which combines the work of numerous developers.

Key Concepts:

- Repository (Repo): A collection of files and their version history.
- Commit: A snapshot of changes at a specific point in time, with a unique identifier.
- Branch: A parallel line of development allowing isolated work on features or bug fixes.
- Merge: Combining changes from different branches into a single branch.
- Pull Request (PR): A proposal to merge changes, often used in Git hosting platforms.

How Git Works:

- Local Development: Developers work on their local machines within a Git repository, making changes to files.
- Staging Area: Changes are staged before committing, allowing selective inclusion in the next commit.
- Commits: Developers commit changes, creating a snapshot of the project with a message describing the modifications.
- Branching: Branches enable parallel development without affecting the main codebase.
- Remote Repository: Developers push commits to a remote repository (e.g., GitHub), making changes accessible to others.
- Pulling Changes: Developers pull the latest commits from the remote repository to stay up-to-date.
- Merging: Merging combines changes from different branches, automatically handled by Git in most cases.
- Collaboration: Git facilitates collaboration through branches, pull requests, and easy sharing of code changes.

RESEARCH TASK

Research: Online Code Collaboration Tools
Research an open-source software project. Determine and describe the online code collaboration tools used by the developers.

Application Programming Interface (API)

An API is a connection or interface between two applications. An API call or request, which includes the input data, is sent to the remote system. A response is returned from the remote system. Depending on the call, the response may simply indicate the call was successful or the response may return information retrieved from the remote system.

For example, Wordpress includes a selling system known as WooCommerce. Developers can write API calls that retrieve customer orders from an online Wordpress based store. The order can then be included in the warehouse database to enable product distribution.

Some other API examples include:

- 7/11 petrol stations allow you to order the cheapest petrol price for all of the stations in a 5km radius from your house. You can access and use 7/11 APIs and integrate them into your own projects. A simple example could be to simply publish the best prices in the country or state for petrol online.
- The Stack Overflow API gives you access to their repository of information that can be customised and displayed via a web browser to suit your needs.
- Most software-as-a-service (SaaS) providers offer APIs that let developers write code that posts data to and retrieves data from the provider. The Google API is able to provide data on up-to-date traffic information to ride sharing applications which allows clients to customise maps for their websites.
- Developers can use multiple different programming languages to create web-based APIs, including Java, JavaScript, Perl, Python, and Ruby. Each call that is a part of these APIs has a defined syntax, and each vendor that provides an API, documents its syntax, usually on their site or sometimes on sites like GitHub.

CLASS ACTIVITY

Research Task: APIs
Each member of you class is to spend 20 minutes researching an API. Briefly share your findings with the class.

Webhooks

Webhooks are a way for one application to provide other applications with real-time information. They allow one application to send a notification to another application when a certain event occurs rather than constantly looking for new data. This can help save on server resources and costs. A webhook is different to an API as data flows in one direction only rather than both directions.
E.g. Reminders for upcoming client meeting that is sent directly to your calendar.

Dynamic Link Libraries

A dynamic-link library (DLL) is a compiled module that contains functions and data that can be used by another application.

Legacy Software

The term Legacy Software refers to outdated software that is still in use years after it was produced. The system still meets the needs it was originally designed for however, it doesn't allow for growth. What a legacy system does now for the company is all it will ever do. Programmers often need to write new software that will integrate with Legacy software so that the company can implement new software systems.

It often surprises people to learn that the underlying code for many major systems has been in use for many decades. The task of rewriting the legacy code can be enormous which must be weighed against the ongoing maintenance of old code.

CLASS DISCUSSION

Brainstorm: Software
Brainstorm examples of software systems you are aware of that integrate with other software systems. Can you identify the nature of the connection that allows such data integration?

Testing and Debugging

Testing and debugging are integral to all stages of the software development cycle. In this stage the focus is on higher levels of testing that take place after the project has been coded in a programming language. Personnel within the development process perform alpha testing with real data. Beta testing occurs when the product is distributed for use to a limited number of outside users. These users are engaged to report any faults or recommendations back to the software development company.

Key Term: Alpha Testing
Testing of the final solution by personnel within the software development company prior to the product's release.

The testing process is central to a software development company's quality assurance. The aim is to ensure the product meets the original requirements and specifications. In terms of quality assurance, we also evaluate the success of the design and development processes. Another purpose of testing is to evaluate the product's performance against recognised industry standards or benchmarks.

Key Term: Beta Testing
Testing of the final solution by a limited number of users outside the software development company using real-world data and conditions.

We discuss levels of testing, from unit or module level testing, through program testing and finally system testing. The generation of test data for specific purposes is discussed. Live test data is produced to test large file sizes, different transaction types, response times, volume data, and to ensure processing is correct. Software tools are used to automate and report on the testing process.

The results of the testing process are used to provide feedback so problems encountered can be corrected before the product is finally released.

Comparison of the solution with the specifications and requirements

The original specifications and requirements, which may have been modified during the development of the product, must be tested for compliance. The aim of the project is to implement the full set of requirements. Because the requirements are written in such a way that they can be readily tested, a pass or fail can be allocated to each requirement. A checklist of all requirements is used. Each requirement is tested in turn. If faults are encountered, then that aspect of the product is altered until the requirement is met.

Specifications were developed as a framework for the development process. Each of these specifications needs to be evaluated against the final product. Many of the specifications will involve internal issues; the results of the testing process will uncover inconsistencies in approach that have occurred during development. These inconsistencies must be identified and rectified both in the existing product and for future products.

Levels of testing

Software products can be tested at various levels. Each individual subroutine and module within a program are tested as it is created. Each program will be tested to ensure modules work together correctly. During the development stage, we were interested in the operation and processing of the actual programming code. We now need to consider broader system-level testing.

What is system-level testing?

System-level testing aims to ensure that the hardware, software, data, personnel and procedures that form the components of the final system, are able to work together efficiently, correctly and in the manner intended with the new software product. Not only does the software need to operate correctly, but it must also operate correctly within the various system environments in which it will be installed and used. To achieve this aim requires the use of real test environments utilising live test data.

CLASS DISCUSSION

Discusssion: Beta Testing Questionnaire
A new website has been developed by an office supply company. The company traditionally sells most of its goods by mail order. The new website allows purchases to be made online. The product is now at the beta testing stage.

Describe the type of businesses the office supply company may choose to be part of their beta testing program. Compile a questionnaire that could be used to evaluate the success of the new website in terms of issues related to personnel and procedures.

Use of live test data for system testing

When a system is finally installed and implemented within the total system it is said to be 'live'. Once a product goes live it needs to undergo a series of tests to ensure its robustness under real conditions. This testing occurs using live test data. For most products live test data should be created to test each of the following conditions:

- Large file sizes
- Mix of transaction types
- Response times
- Volume data (load testing)

Large file sizes

Many commercial applications obtain input from large databases. During the development of software products, relatively small data sets are used. At the alpha and beta testing stages large files should be used to test the system's performance.

The use of large files will highlight problems associated with data access. Often systems that perform at acceptable speeds with small data files become unacceptably slow when large files are accessed. This is particularly the case when data is accessed via networks.

Large data files highlight aspects of the code that are inefficient or require extensive processing. Many large systems will postpone intensive processing activities until times when system resources are available. For example, updating of bank account transactions takes place during the night when processing resources are available.

CLASS DISCUSSION

Discussion: Large File Testing

A rental car company is implementing a new computer system across its national network of 1200 branches. The branches are linked via a network of computers. Because of the remoteness of many branches, communication links are unreliable hence data must be held locally at each branch. Because cars can be returned to any branch, each branch requires a record of all rentals from all branches.

A system known as data replication is used. Essentially, every branch has a copy of the entire database. Changes to a particular record at one branch are duplicated across all branches every 10 minutes. The rental car company has a fleet of some 7000 vehicles with approximately 4000 new rental transactions occurring each day.

1. Each rental transaction is held for five years. How many transactions should be in the test file to ensure performance remains satisfactory?
2. What types of problems are likely to be highlighted as a result of testing the product with such a large file? Explain.

Mix of transaction types

Testing needs to include random mixes of transaction types. Module and program testing usually involve testing specific transactions or processes one at a time. During system testing we need to test that transactions occurring in random order do not create problems.

Often the results of one process will influence a number of other processes. If a transaction is currently being completed on specific data and that data is altered by another transaction, then problems can occur. When a number of applications are used to access the same database then conflicts are inevitable. Software must include mechanisms to deal with such eventualities.

CLASS DISCUSSION

Discussion: Banking Transactions
Banking systems involve a multitude of applications that access individual account details. Some accounts may experience many thousands of transactions per hour. This is particularly the case with public bodies such as the tax office and social security. Deposits and withdrawals are taking place at an enormous pace. These transactions originate from a large variety of sources: ATMs, bank branches, post offices, the internet, etc.
New applications that interface with existing banking systems need to be able to deal with this situation. Transactions must be absolutely secure given the nature of the application.

1. Describe possible scenarios that could result in problems when multiple transactions attempt to access a single account's details.
2. What type of live test data could be used to test the correctness of these banking transactions?

Response time

The response time is the time taken for a process to complete. The process could be user activated or it could be activated internally by the application. Response times are dependent on all the system components, together with their interactions with each other and other processes that may be occurring concurrently.

Any processes that are likely to take more than one second should provide feedback to the user. Progress bars are a common way of providing this feedback. Data entry forms that need to validate data before continuing should be able to do so in less than one second; 0.1 second is preferable. Studies have shown that users will tolerate delays of up to one second, however their concentration will deteriorate if the delays are frequent. Response times of around 0.1 second give the user the impression of an instantaneous response - this is of course the ideal situation.

Response times should be tested on minimum hardware using typical data of different types. Any applications that affect and/or interface with the new product should be operating under normal or heavy loads when the testing takes place.

CLASS DISCUSSION

Discussion: Website Response Times
On the internet, response time rules! Sites that include animations and complex graphics often reduce the effectiveness of the site by increasing response times. The time taken to download these graphics can often cause prospective customers to give up and go to a faster competitor site. There is money to be made by closely examining the response times for each transaction on a new website.

There are many companies that will analyse the response time for different aspects of websites. Their findings can result in significant savings for their clients. One company was founded by an individual who advertised his business with the slogan 'I'll tell you why your website is rubbish'. His business was based on response time analysis.

1: Explain why response times on websites are so crucial to the success of these sites.

2: Describe the nature of tests that could be implemented to test the response times for websites. Justify your answers.

Volume data (load testing)

Large amounts of data should be entered into the new system to test the application under extreme load conditions. Multi-user products should be tested with large numbers of users entering and processing data simultaneously. Large systems that require extensive data input require special consideration.

Alpha testing by the software developer must try to simulate the number of users who will simultaneously use the product. This can be done using a laboratory of computers and users in conjunction with automated software tools. In many cases, it is difficult to simulate real volumes of data in a beta testing environment without actually implementing the system.

Tools are available that enable automatic completion of data entry forms. Many of these software tools allow the creation of virtual users. This allows one machine to simulate the activities of hundreds or even thousands of simultaneous users entering data.

CLASS DISCUSSION

Discussion: Load Testing
The post office is implementing a new application in its existing computer system. This application allows businesses to purchase and print stamps via the internet. It is envisaged that some 20,000 businesses will take advantage of this initiative within one year of its introduction. Within five years, some 150,000 businesses will be using the service.

The software has been written and is about to undergo volume testing. It is not possible to beta test the product with 150,000 users, so a simulation using software automation tools is used.

1: The volume of data to be processed is crucial to the success of this product. How can volume testing be undertaken when the customer base does not at present exist?

2: The security of these transactions is crucial. Discuss methods that could be used to test the security of the system, both in terms of payment for stamps and in terms of ensuring printed stamps are not counterfeit.

Installation

Once a new software product has been produced it must be installed and then implemented on site. There are a number of methods of introducing a new system and each of these methods suits different circumstances. Usually installation will involve conversion from an old system that the new system is designed to replace, so often these methods are known as methods of conversion.
The four common methods of installation (or conversion) are:

- Direct cut-over
- Parallel
- Phased
- Pilot

Direct cut-over

This method involves the old system being completely dropped and the new system being completely installed at the same time. The old system is no longer available. As a consequence, you must be absolutely sure that the new system is totally functional and operational. This conversion method is used when it is not feasible to continue operating two systems together. Any data to be used in the new system must be converted and imported from the old system. Users must be fully trained in the operation of the new system before the conversion takes place.

CLASS DISCUSSION

Consider: ATM Installation
A bank is introducing a new Automatic Teller Machine into many of its suburban branches. This new ATM includes a colour screen, which is also a touch screen. The software that controls the ATM has been thoroughly tested. A direct cut-over method of installation is really the only method open to the bank. It would be impracticable to operate the old and new ATMs side by side. Even if this were possible, it would be ridiculous to expect customers to enter their details into both machines.

Discuss: ATM Installation
List and describe any potential problems you can envisage that may result from the sudden installation of these new ATMs. Think about issues from the point of view of both the bank and its customers.

CLASS DISCUSSION

Consider: Ticketing System
A circus, which travels throughout Australia, is changing its ticket selling system from a manual system to a computerised system. Patrons purchase their tickets at a ticket booth just before they enter the big tent.

Discuss: Ticket System Installation
Is a direct cut-over conversion method appropriate for the above scenario? Explain your answer. Do all functions of the ticketing system need to be converted using the direct cut-over method? Discuss.

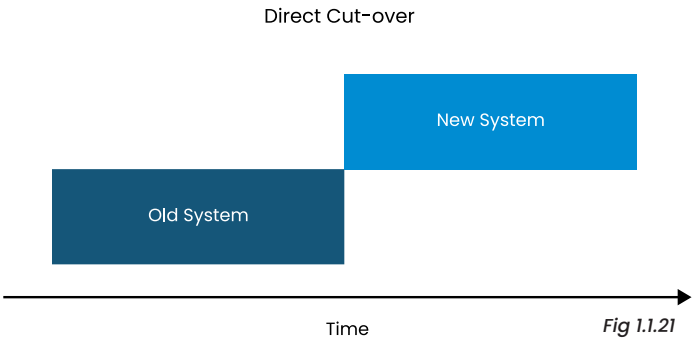


Fig 1.1.21

Parallel

The parallel method of installation (or conversion) involves operating both systems together for a period.

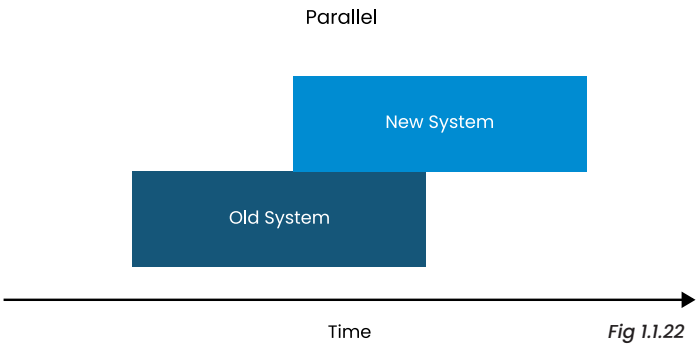


Fig 1.1.22

This allows any major problems with the new system to be encountered and corrected without the loss of data. Parallel conversion also means users of the system have time to familiarise themselves fully with the operation of the new system. In essence, the old system remains operational as a backup for the new system. Once the new system is found to be meeting requirements then operation of the old system can cease. The parallel method involves double the workload for users as all functions must be performed on both the old and the new systems.
This method of installation is especially useful when the product is of a crucial nature. That is, dire consequences would result if the new system were to fail. By continuing operation of the old system, the crucial nature of the data is protected.

From the software developer's point of view, parallel conversion can mean that both the old system and the new, upgraded system are operating at the same time but at different sites. Each individual site may have used a direct cut-over method of conversion. This situation is particularly common when the system is used by a large number of customers and they must be given time to commence operations using the new system. With a large customer base it will take some time to upgrade each site to the new product. A consequence of this situation is that the developer must continue to support previous versions of their product until the conversion is completed.

CLASS DISCUSSION

Consider: Mobile Networks
The digital mobile phone network is now the only type of network available in large Australian cities. Digital mobile networks were introduced in Australia in the early 1990s, however the old analog mobile networks were only taken out of service in the late 1990s. Both systems operated together for some 5-10 years.

Discuss:
Why was it necessary for both analog and digital mobile phone services to be maintained for so many years?

Discuss:
Many users were happy with the old analog networks. Why do you think the mobile phone service providers decided to switch to digital networks?

CLASS DISCUSSION

Consider: New Hospital Systems
Five new computer-controlled life support systems have been purchased by a hospital. These life support systems are able to monitor a patient's temperature, blood pressure and various other vital signs. When an irregularity is detected, the medical staff are alerted electronically. Medical staff continue to manually monitor each patient's vital signs and record them on a paper chart on the end of each patient's bed.

Discuss:
Is it necessary for a patient's vital signs to continue to be manually monitored? Explain your answer.

Discuss:
The hospital decides it is no longer necessary for the manual system to remain in place. How would you feel about this decision if you were one of the patients, or a close relative or friend of one of the patients?

Phased

The phased method of installation (or conversion)from an old system to a new system involves a gradual introduction of the new system whilst the old system is progressively discarded. This can be achieved by introducing new parts of the new product one at a time while the older parts being replaced are removed.

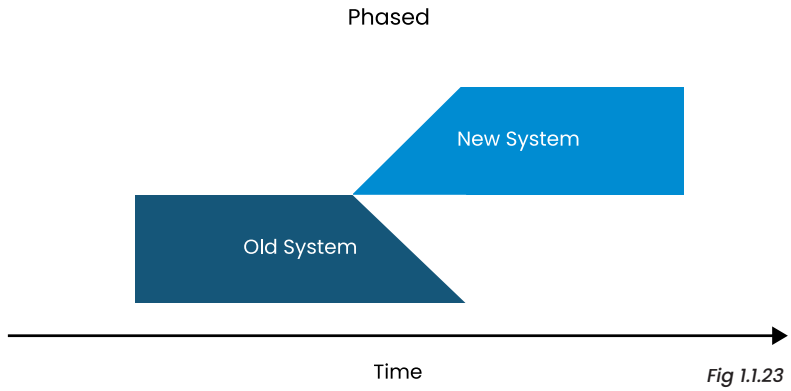


Fig 1.1.23

Often phased conversion is used because the product, as a whole, is still under development. Completed modules are released to customers as they become available. Phased conversion can also mean, for large businesses, that the conversion process is more manageable. Parts of the total system are introduced systematically across the business, each part replacing a component of the old system. Over time the complete system will be converted.

CLASS DISCUSSION

Consider: Chemsoft Installation
A software development business, called Chemsoft, specialises in software to support the operations of pharmacies. They currently have around 4000 chemists using their product. Chemsoft constantly works on upgrading components of their product. As each component is completed, it is distributed to chemists for installation. Component upgrades are sent around 3 times per year. A full upgrade of the product usually takes about 18 months.

Discuss:
It is not possible for Chemsoft to distribute and install each component at all sites at the same time. Discuss methods Chemsoft could use to implement the upgrade of each component. Assume nobody from Chemsoft has to be physically onsite to perform the installation.

Discuss:
Some chemists never quite get around to performing some of the upgrades. What sort of problems would this cause for the software developer Chemsoft? How could Chemsoft encourage their clients to implement the upgrades?

CLASS DISCUSSION

Consider: Large Restaurant
A large restaurant is introducing a new computer system. There are essentially four software modules that interface together to operate the functions of the restaurant: point of sale, accounting, wages and ordering/stocktaking.

Discuss:
If the restaurant decides to use a phased conversion method, how would this be achieved? Justify the order of installation for each of the four modules that make up the new package.

Discuss:
As part of the conversion the staff need to be trained to use the new system. The restaurant currently employs a total of 4 managers, 15 kitchen staff and some 20 table staff. Suggest how the training of all these staff members could be accomplished in an economical and efficient manner.

Pilot

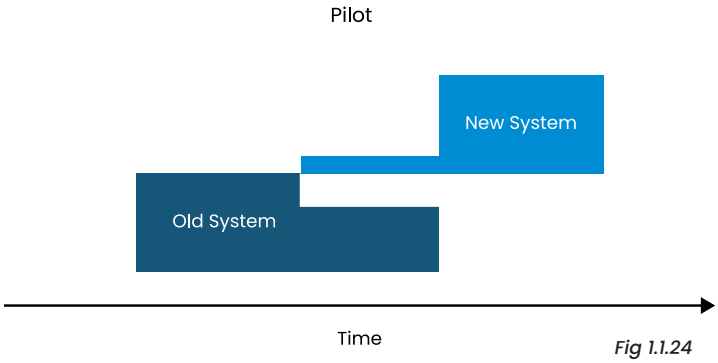


Fig 1.1.24

With the Pilot method of installation (or conversion), the new system is installed for a small number of users. These users learn, use and evaluate the new system. Once the new system is deemed to be performing satisfactorily then the system is installed and used by all. This method is particularly useful for new products, as it ensures functionality is at a level that can perform in a real operational setting. The pilot method also allows a base of users to learn the new system. These users can then assist with the training of others once the system has been fully installed. The pilot conversion method can be viewed as the final testing of the product. Both the developer and the customer are able to evaluate the product in an operational environment prior to its full installation.

CLASS DISCUSSION

Consider: Email Software
A large company, with some 300 employees, is intending to change its Email software. They decide on a new Email software package. The package is installed and used by 15 employees for 3 months. Following this trial (or pilot) period, the new Email system is implemented on all employees' machines.

Discuss:
Why do you think the company decided to trial the Email package with 15 employees? Explain your answer.

Activtity:
The 15 employees used for the trial were carefully chosen. Make up a list of criteria the company management may have used to select these 15 employees for the trial.

CLASS DISCUSSION

Consider:
A large high school has an outdated system to create reports for parents. They wish to improve the efficiency of their reporting and also to enhance the look of the reports. They decide on a commercial school reports package that seems to fulfil their requirements.

Discuss:
How could the school best pilot this new package? What sort of criteria could they use to evaluate the success of the pilot?

Many teachers at the school are reticent to engage with the new system. How could the pilot method be used to increase the acceptance of the teachers at this school?

EXERCISE 1.1.3 – MULTIPLE CHOICE

1. A recipe includes a list of ingredients and a method describing how to make the dish. Which of the following is the best analogy to designing software?

   A. The ingredients are the algorithm and the method is the data structure.

   B. The ingredients are the data items and the method is the algorithm.

   C. The ingredients are the data structure and the method is the algorithm.

   D. The ingredients are the data and the method is the data structure.

2. Which of the following best distinguishes between data types and data structures?

   A. Data structures refer to the category of data such as integers, strings, or Booleans. On the other hand, data types refer to the organisation of data to allow for efficient storage and manipulation.

   B. Data types refer to the category of data such as integers, strings, or Booleans. On the other hand, data structures refer to the organisation of data to allow for efficient storage and manipulation.

   C. Data types refers to the actual data, the values themselves, such as 4, 6.7. "Egg" or True. A data structure, on the other hand, is the container that stores the data types within the computer memory both for storage and whilst processing.

   D. A data type is the category of data relevant to the user such as a sound recording, list of marks, text of a short story. On the other hand, data structures refer to the organisation of data to allow for efficient storage and manipulation.

3. At which software development step does coding take place?

   A. Design

   B. Development

   C. Integration

   D. Testing and debugging

4. Which acronym refers to a tool used by teams of programmers to manage source code changes, including backups and version control?

   A. JIRA

   B. TDD

   C. Git

   D. VCS

5. The ability for one system to use the services of another enables efficient integration of software systems. Which of the following is NOT an example of an integration technology?

   A. API

   B. Webhook

   C. DLL

   D. Github

6. Which term best describes in house testing prior to release to a wide audience?

   A. Alpha testing

   B. Beta testing

   C. Debugging

   D. Acceptance testing

7. At which stage of the software development process does testing to ensure code operates as expected occur?

   A. Development

   B. Integration

   C. Testing and debugging

   D. All of the above

8. A new software product is installed at a small number of sites. Once it is operating well, the remaining sites are converted to the new system. Which method of implementation was used?

   A. Direct cut-over

   B. Parallel

   C. Phased

   D. Pilot

9. A poultry farm installs a computerised feeding and watering system throughout all its sheds. The owners have viewed the same system successfully operating on a number of other poultry farms. Which method of implementation would most likely be used?

   A. Direct cut-over

   B. Parallel

   C. Phased

   D. Pilot

10. The energy authority in a large city is implementing a new computer system throughout their branches. They decide to implement parts of the system progressively over a two-year period. Which method of implementation is being used?

    A. Direct cut-over

    B. Parallel

    C. Phased

    D. Pilot

EXERCISE 1.1.3 - SHORT ANSWER

11. Write an algorithm in the form of a list of steps that explains how to add two 3 digit numbers together. Have another person test your list and note any errors they make, then correct the list of steps as best you can.

12. Generally, currency data types are more accurate than floating point real data types. Research and explain why this is the case and briefly outline the differences.

13. Research simple examples of Application Programming Interface (API) examples and describe at least two, with examples. For instance, entering the following URL into a browser https://dog.ceo/api/breeds/image/random requests the URL for a random dog image. The response is a URL to an image like the ones below.



14. Describe the advantages and disadvantages of the direct cut-over method of installation compared to the other three methods.

15. Investigations following a major air accident result in the rewriting of a software-based backup system that controls the flow of fuel to a jet's engines if the main system fails. The problem is unlikely to cause future accidents, however conversion is made mandatory for all airlines. The regulatory body controlling air safety needs to develop an installation strategy that ensures all aircraft are converted. Suggest, explain and justify a possible strategy.

Maintenance

Maintenance is an ongoing process of correction and refinement. Many software products have a life span of many decades. These products will require maintenance for them to continue to meet the expectations of their users. Maintenance includes the correction of errors (bugs) in the source code together with upgrades to enhance the functionality of the product to meet new or changing requirements.

We will examine the processes involved in maintaining a software solution and the methods of identifying and determining what changes should be made. This process involves ongoing communication with the product's users. Once a decision has been made to modify the product, the software development cycle is commenced. Finally, a method of distributing the modification is devised, and the modification is installed for use. For example, small bug fixes may be distributed as a patch, whereas large changes to the user interface may require a complete upgrade to be distributed.

Documentation of the original product's development will prove invaluable to maintenance programmers. Maintainability is a measure of how easily a solution can be maintained; it is closely linked to the quality of the original documentation. Often the maintenance programmer will not be the same as the original programmers. Many software development companies have a separate team of maintenance programmers. The task of maintaining a product is simplified when the original technical documentation is thorough and consistent.

Modification of code to meet changed requirements

Software solutions are not static entities - they evolve over time. Most applications mature as new versions are introduced. These new versions will correct bugs and introduce new functionality not present in the original product. Changes made to subsequent releases of applications are often the result of user requests. Many software companies provide facilities for users to make suggestions in regard to new functionality or changes to existing functionality. Remember, fulfilling the requirements of end users is a central aim for most software products.

Identification of the reasons for change in source code

The possible reasons for changing source code are many and varied. Reasons for change common to all applications include bug fixes, new hardware and new software purchases. Major upgrades of operating systems will have a flow-on effect to other applications. Upgrading of applications to take advantage of features available in new operating systems is required.

A software company whose products have a large customer base may receive many thousands of requests for modification of their products. The company must then prioritise these requests and make decisions on which modifications will be included and which will not. Often registered users are surveyed to obtain their views on possible modifications. Support departments provide a valuable source of data: the number of support issues about specific functions highlights useability problems. This information can be used as the basis for selecting appropriate modifications for inclusion in an upgrade.

Customised software written for a specific client will require continual maintenance as requirements change. As customised software is an expensive purchase, maintaining the product is normally preferable to changing products, or developing a new product from the ground up. Because custom software is written to solve a particular problem, minor changes to the requirements often necessitate alterations and additions to the source code.

CLASS DISCUSSION

Discussion: Multimedia Player
A multimedia title uses Microsoft's Media Player to play video sequences within the product. A new version of Media Player has been released that incorporates a more graphically rich set of user controls. A multimedia title, although still operable, does not contain sufficient space for the new Media Player interface.

1: "Microsoft should not make changes to products that impact other products. In this case they have done so, therefore they should be responsible for any costs incurred in modifying the multimedia title." Do you agree with this statement?

2: The author of the multimedia title decides to correct the problem and sends out a patch to all registered users. How could they correct the problem so that future upgrades to Media Player will not affect the operation of their product?

CLASS DISCUSSION

Context: Surgery Maintenance
A software development company produces and distributes a product that automates record keeping in doctors' surgeries. As part of each surgery's support contract, the software company provides an annual upgrade of the product. The software support team maintains a database of all their support calls. This database is used as the basis for identifying modifications to be included in each upgrade. The following table lists the ten most common support issues.

Issue	Frequency	Issue	Frequency
Need to link members of the same family.	65	Cancelled appointments are not made available immediately.	29
No report available to list frequency of drug prescription by doctor (new government legislation)	59	Need to minimise application so other programs can be used withoutexiting.	24
Standard HP 6L printer driver not compatible.	58	Individual patient records can only be viewed on one terminal at a time.	18
Scheduling of doctors does not take account of multiple surgery locations within the one practice.	38	Program crashes when two referrals are printed for one patient	17
Unable to import drug references from new Department of Health Version 7.1 database.	31	Loss of network connection is not notified until a save operation is initiated.	15

Fig 1.1.25

Discussion:
Categorise each of the above ten issues as either bugs in the application, hardware issues, other software issues or changes to program requirements. Explain why you have placed each issue into a particular category.

Discussion:
The above ten issues need to be prioritised. Should the frequency or number of times the issue has arisen be the sole criterion for deciding on its inclusion in the upgrade? What other criteria should be used? Decide on an appropriate order of priority for these ten items.

Location of the section to be altered

Once a decision has been made to modify a particular aspect of a product, the location of the section to be altered needs to be determined. Models of the system are used to assist in this process. Structure charts and data flow diagrams will assist in the location of the module or modules requiring modification. Once a module has been identified, the programmer can analyse the original algorithms and models, and the actual source code. A thorough understanding of the original source code is required before any modifications are undertaken.

CLASS DISCUSSION

Discussion: Custom Book Reseller Program
A decision has been made to modify a customised book reseller program. One of the modifications required involves automatically reordering a predetermined number of a specific title when the stock held is insufficient to fulfil an order. Currently only the required number of books is ordered if insufficient stock is held in the warehouse. The programmer employed to complete this task is currently analysing the original data flow diagrams used to model the system. The level one data flow diagram below was produced by the original developers of the product.

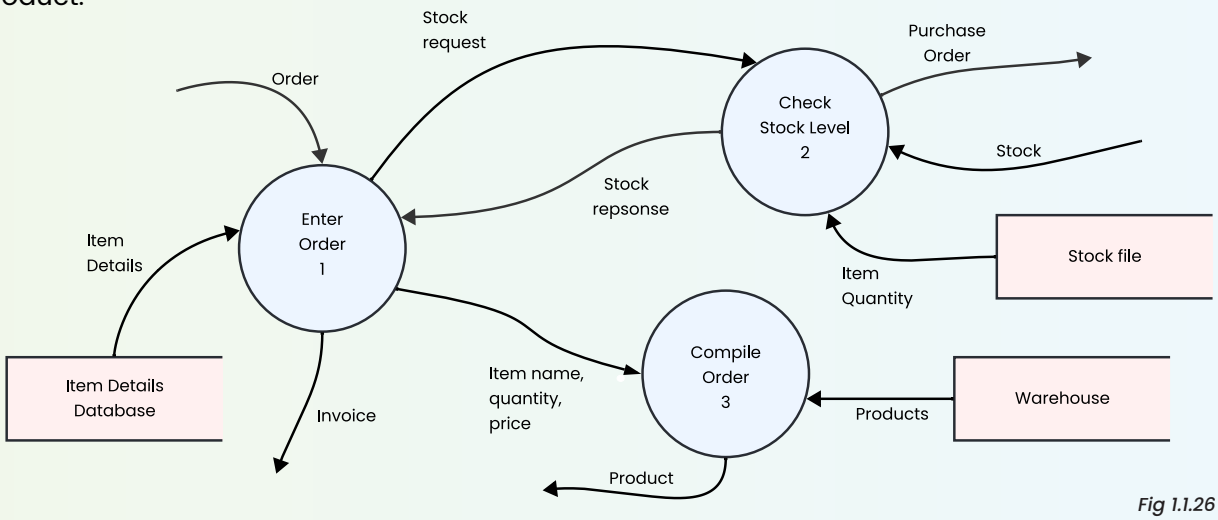


Fig 1.1.26

The programmer determines that the relevant code is held in the Check Stock Level module. The data flow diagram for this module is reproduced below:

After analysing this data flow diagram, the programmer decides to investigate module 2.2 dealing with insufficient stock. This module seems to generate the data for the purchase order to the supplier. This seems the most likely module in which to implement the desired modification.

Discuss:
Why do you think the programmer has selected module 2.2 for possible modification?

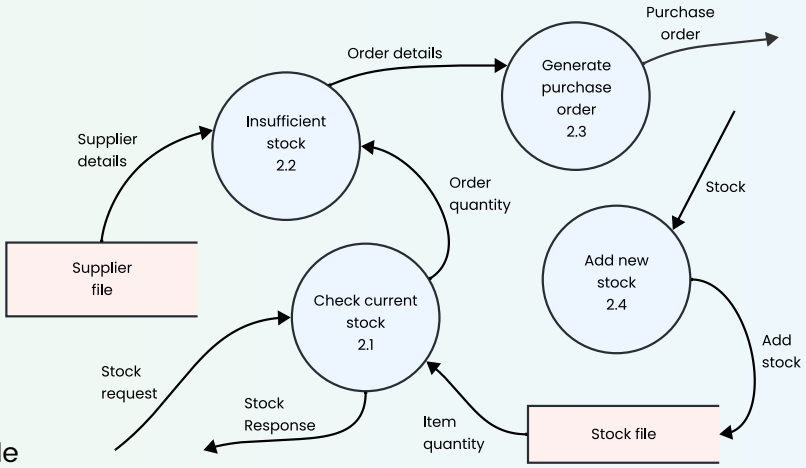


Fig 1.1.27

Determining changes to be made

After the section of code to be modified has been located, we need to determine the changes to be made. Depending on the nature of the modification, changes may be required to data structures, files, and the user interface, as well as to the source code.

The consequences of changes made to one module must be considered. If a record data structure is altered then what effect will this have on other modules that access this data? For example, if a field within a record that once contained Boolean data is changed to hold integer data then there will be consequences for all modules that access these records. We don't want to solve one problem only to cause a number of new ones. Analysis of the original documentation should alert programmers to the possible consequences of changes.

Implementing and testing the solution

Once the changes to be made have been determined, these changes must be implemented within the existing application. In many cases, changes will be made to the source code and the application will be recompiled, tested, and distributed. In other cases, a small application or patch may be written to implement the changes on end users' systems. Custom systems can often be changed directly on the site or over an internet link. This is particularly the case with script and macro modifications.

The implementation of modifications can have a ripple effect to other aspects of the application. These problems are minimised if the original product was designed using a structured modular approach. When each procedure or function has been designed independently then the effect of changes will be easier to identify and correct. Modules that are used in several places throughout the application should be modified in such a way that they retain their original processing, including input and output parameters. Changes to modules that alter parameters will require modifications to each higher-level calling module.

Testing of modifications should be performed using the same techniques employed for the development of new products. Test data used during the original testing can be reused. The tests should not be restricted to the modified code segments, but applied to all aspects of the application that are in any way affected by the change.

EXERCISE 1.1.4 – MULTIPLE CHOICE

1. There are a number of ways of distributing and installing modified software. A piece of code inserted into an existing executable program to correct a problem is called a(n):

   A. upgrade.
   B. patch.
   C. script.
   D. bug fix.

2. Maintenance of software solutions is necessary to:

   A. correct bugs in the source code.
   B. modify the program to reflect changing requirements.
   C. ensure software solutions remain compatible with other technologies.
   D. All of the above.

3. In general, for large software products, maintenance programmers:

   A. are usually members of the original development team.
   B. control the configuration management of the system.
   C. are rarely the same as the members of the original development team.
   D. form part of the customer support team.

4. Modifications to particular procedures and functions should aim to:
   A. maintain the original functionality of the code whilst introducing new features.
   B. have few or no side effects on any routines that call the procedure or function.
   C. allow existing data files to operate correctly once the modification is implemented.
   D. All of the above.

5. Modifications to individual routines within software products:
   A. can safely be tested in isolation from the remainder of the application.
   B. require minimal testing because they are unable to affect other modules.
   C. should only be made when there is sufficient demand for the modification from customers.
   D. require testing in the same way as the original module.

6. Identifying the section of code to be altered is simplified when:

   A. the original algorithms are available.
   B. system models are accurately maintained.
   C. the original test data is retained.
   D. All of the above.

7. There are many reasons why software products require modification. Possible reasons include all of the following EXCEPT:

   A. To correct bugs in the product.
   B. Changes to user requirements.
   C. Upgrades to operating systems.
   D. To improve face-to-face training.

8. Modifying software products generally involves more than altering the source code. Other possible modifications include all of the following EXCEPT:

   A. Alterations to documentation to reflect the changes.
   B. Changes to data structures.
   C. Examining competitors' products.
   D. Creation or modification of testing and related test data.

9. The maintainability of a software solution can be best described as:

   A. the usefulness of a software solution in regard to fulfilling users' requirements.
   B. how easily a software solution can be maintained.
   C. the speed at which modifications can be implemented into a software solution.
   D. a measure of the ability of the product to evolve over time.

10. Prioritising corrections to be included in a product's upgrade requires consideration of which of the following?

    A. Frequency of error: How many times have users encountered the error?
    B. Impact of the error: Does the program crash? Is data lost or corrupted as a consequence of the error?
    C. Cost and time to correct the error: Is it cost effective to correct? Can it be corrected within a reasonable time?
    D. All of the above.

EXERCISE 1.1.4 - SHORT ANSWER

11. Outline reasons why software products require ongoing maintenance.

12. Explain how structure charts and data dictionaries can assist maintenance programmers.

13. Popular software applications with hundreds of thousands of users attract huge numbers of enhancement and upgrade requests. Identify criteria that should be considered to prioritise which requests should be implemented and in which order.

Use the following scenario to assist in answering questions 14 and 15.

A school's reporting software requires a number of modifications. Firstly, the school wishes to be able to print out the reports for a single year group. Secondly, there are now two different formats for the final report rather than the original, single format. Lastly, each report requires a range of dates indicating precisely the reporting period.

The screen shot at right shows the existing print dialogue for the school's reporting system. Maintenance programmers are currently working on creating the new report format, together with modifications so the range of dates is included on each format.

14. Examine the user interface above. Suggest alterations that should be made to this screen to implement the three changes. Justify and explain your reasons for each modification you suggest.

15. Some time later the school again contacts the software company for further modifications. They wish to be able to preview each report on the screen or print it. The school also has another three different formats they wish implemented. Explain how these modifications can be included on the print screen in such a way that future requests for new formats will not require changes to this screen